



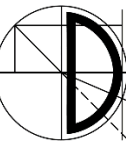
Julius-Maximilians-  
**UNIVERSITÄT  
WÜRZBURG**



# Grundlagen der Programmierung

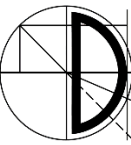
Kapitel 10: Objektorientiertes Programmieren  
Prof. Dr. Samuel Kounev, Daniel Grillmeyer M.Sc.

- Alle Materialien (inkl. Vorlesungs- und Übungsfolien, Vorlesungs- und Übungsaufzeichnungen, Übungsblätter, Programmieraufgaben) in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Inhalte an Dritte ist generell untersagt!
- Vorlesungsmaterialien von Prof. Dr. Detlef Seese wurden als Basis verwendet
- Unterstützung bei der technischen und inhaltlichen Gestaltung des Vorlesungsmaterials leisteten: Martin Sträßer, Daniel Grillmeyer, Jóakim v. Kistowski, Dietmar Ratz, Joachim Melcher, Roland Küstermann, Jana Weiner, Hagen Buchwald, Matthes Elstermann, Oliver Schöll, Niklas Kühl, Tobias Diederich

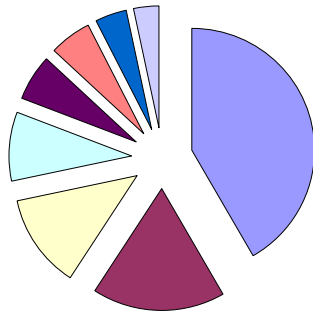


- Treiber der Objektorientierung (historisch)
- Warum Objektorientierung?
- Was ist Objektorientierung?
- Die Philosophie und Grundprinzipien der Objektorientierung
  1. Abstraktion / Generalisierung (abstraction / generalization)
  2. Vererbung (inheritance)
  3. Kapselung (encapsulation)
  4. Polymorphismus (polymorphism)

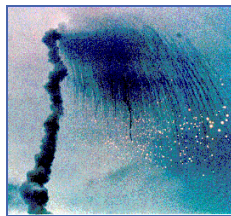
# Die Treiber für die Objektorientierung (Ende der 1980er Jahre)



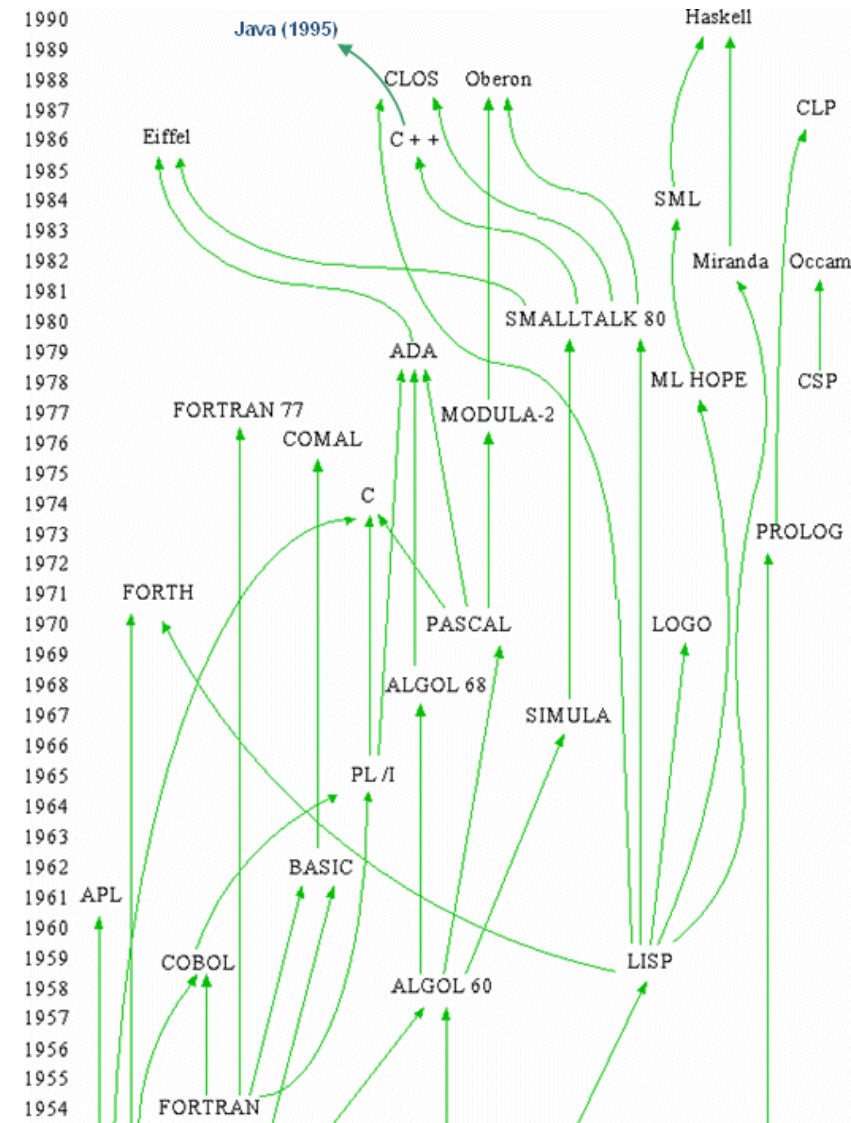
**Grafische Benutzer-Oberflächen** für den neuen PC-Massen-Markt (Programm-Komplexität vervielfacht sich)!



Die **Kosten für die Wartung** der Software sind zu hoch, Anpassungen dauern zu lange und kosten zu viel!



**Softwarekrise** führt zu immer größeren und imageschädigenden Unglücken!





# 1973 Xerox PARC: Der „Alto“ erscheint

- Erster Computer mit graphischer Benutzungsschnittstelle (GUI)  
Vorbild für den legendären Apple Macintosh
- Monochromen Monitor mit einer Auflösung von 606 x 808 Pixel
- Unkodierte Tastatur mit 61 Tasten
- Drei-Tasten-Maus, nutzte Rastergrafik, Fenster, Menüs und Icons
- Über das Ethernet-Protokoll mit anderen Rechnern verbindbar
- Datanaustausch, E-Mail über Intranet
- WYSIWYG-Oberfläche
- Hat die Größe einer Kühltruhe

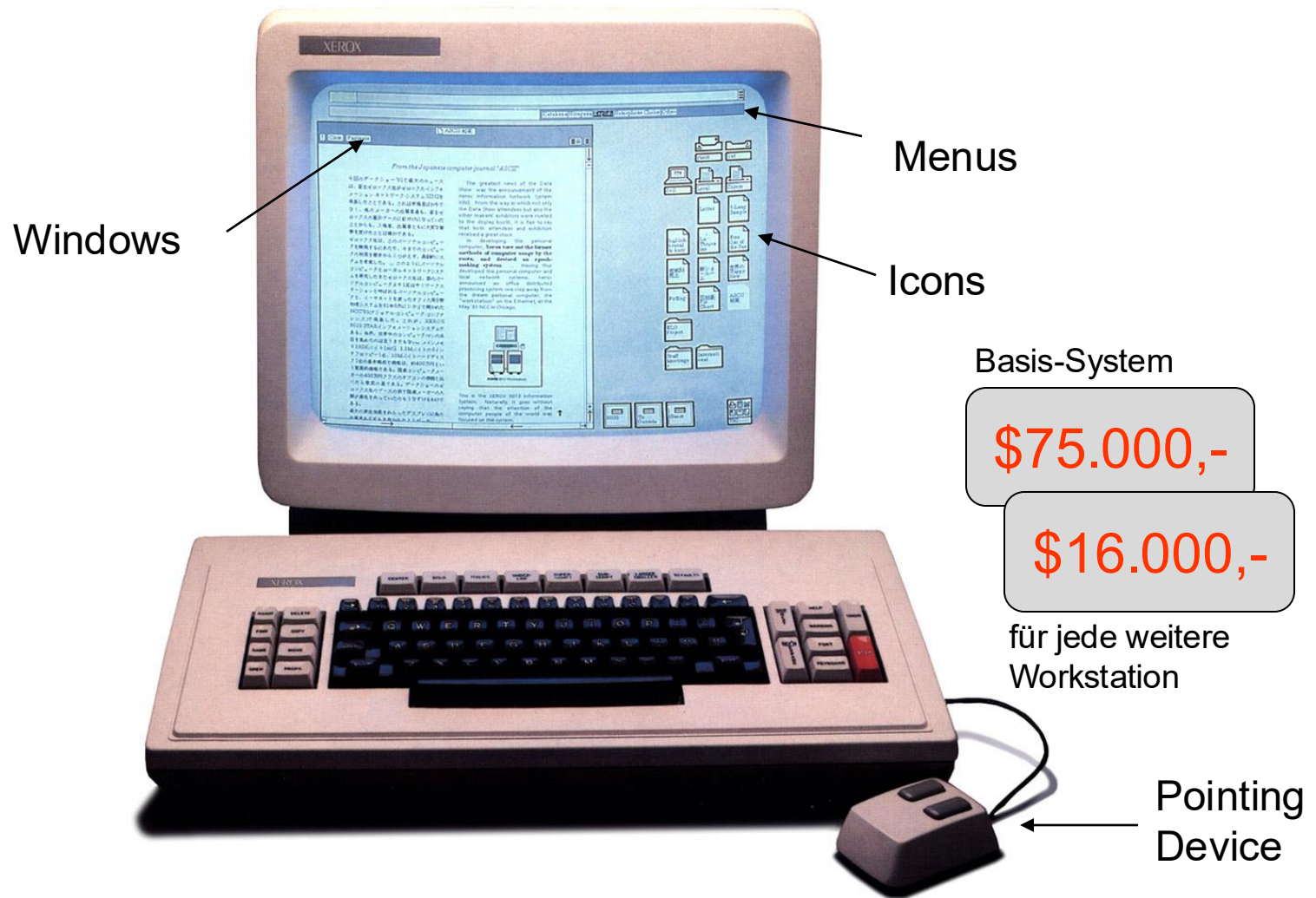
Quelle: [http://de.wikipedia.org/wiki/Xerox\\_Alto](http://de.wikipedia.org/wiki/Xerox_Alto)



**\$32.000,-**

# April 1981: XEROX 8010 „Star“ erscheint

- Diese größenreduzierte Weiterentwicklung des Alto kann als Urvater aller modernen WIMP-Systems (Windows, Icons, Menus, Pointing Devices, WYSIWYG) angesehen werden.



Quellen: [http://de.wikipedia.org/wiki/Xerox\\_Star](http://de.wikipedia.org/wiki/Xerox_Star) und <http://www.digibarn.com/>

# August 1981: Der IBM PC erblickt die Welt

- Der Microcomputer wird zum standardisierten Industrie-Massenprodukt  
Der VW-Käfer der IT ist geboren.



Quelle: [http://de.wikipedia.org/wiki/IBM\\_Personal\\_Computer](http://de.wikipedia.org/wiki/IBM_Personal_Computer) und [http://einestages.spiegel.de/static/topicalbumbackground/23321/charlie\\_chaplin\\_computerverkaeuer.html](http://einestages.spiegel.de/static/topicalbumbackground/23321/charlie_chaplin_computerverkaeuer.html)

# 1984: Apple Macintosh mit Mac OS 1.1

- Der Apple Macintosh – die geniale Verschmelzung des PC-Konzepts mit den sichtbaren Vorteilen des XEROX 8010 „Star“

\$2.495,-

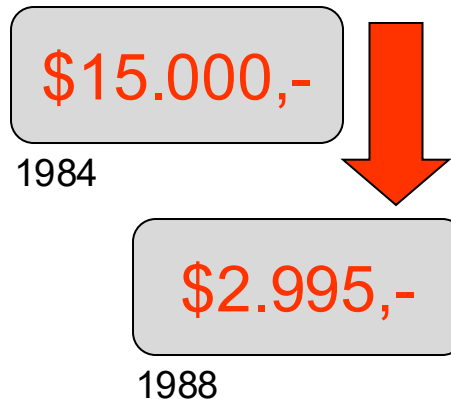


Quelle: <http://oldcomputers.net/macintosh.html>



# August 1984: Der IBM AT und erste Clones sind da!

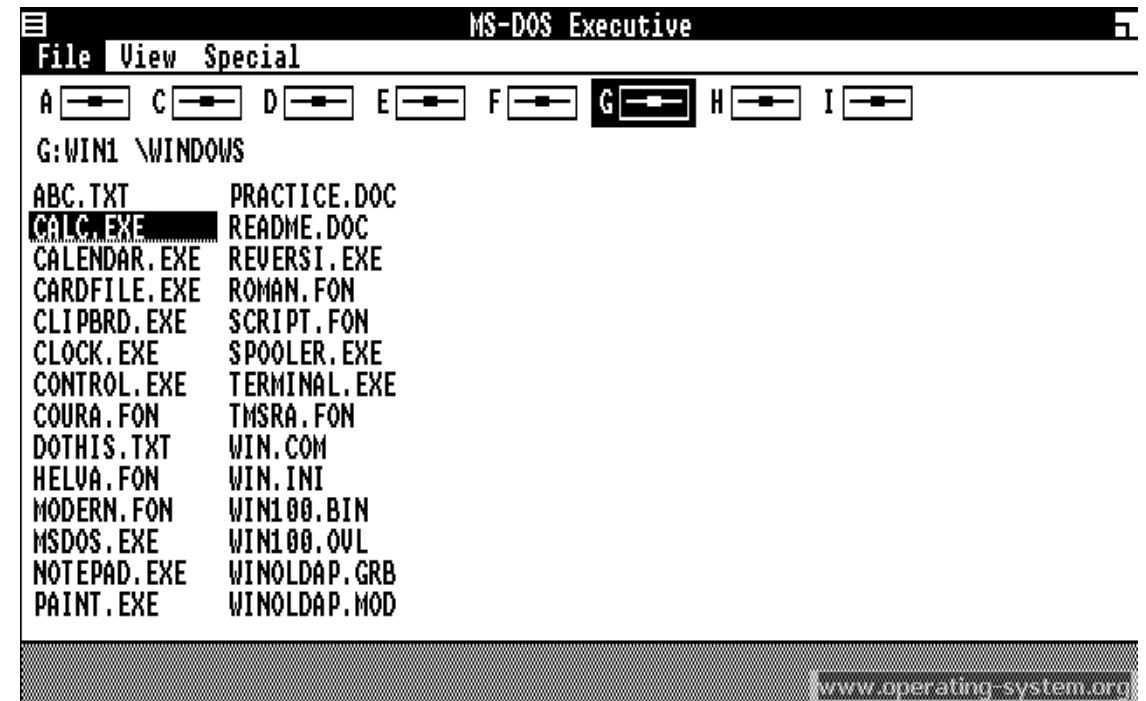
- Drittanbieter wie Hercules (Grafikkarten), Cherry (Tastaturen) und Compaq (Clones) und Microsoft (MS-DOS) nutzen die offene Architektur des IBM PC – und werden zu Global Playern!



Quellen: <http://www.historycorner.de/CoCo3/IBM/ibm5170AT.html> und [http://de.wikipedia.org/wiki/IBM\\_Personal\\_Computer/AT](http://de.wikipedia.org/wiki/IBM_Personal_Computer/AT)

# 1985: Desktop von Windows 1.0

- Grafische Benutzeroberflächen die rein nach dem funktionsorientierten Ansatz erstellt waren, boten wenig Attraktivität



Quellen: <http://toastytech.com/guis/win101executive.gif>, [http://www.operating-system.org/betriebssystem/\\_german/bs-win1.htm](http://www.operating-system.org/betriebssystem/_german/bs-win1.htm) und [http://osdp.bplaced.net/de/screen\\_gallery.php?bsgfx=microsoft/win1/windows1-scr-08.png](http://osdp.bplaced.net/de/screen_gallery.php?bsgfx=microsoft/win1/windows1-scr-08.png)

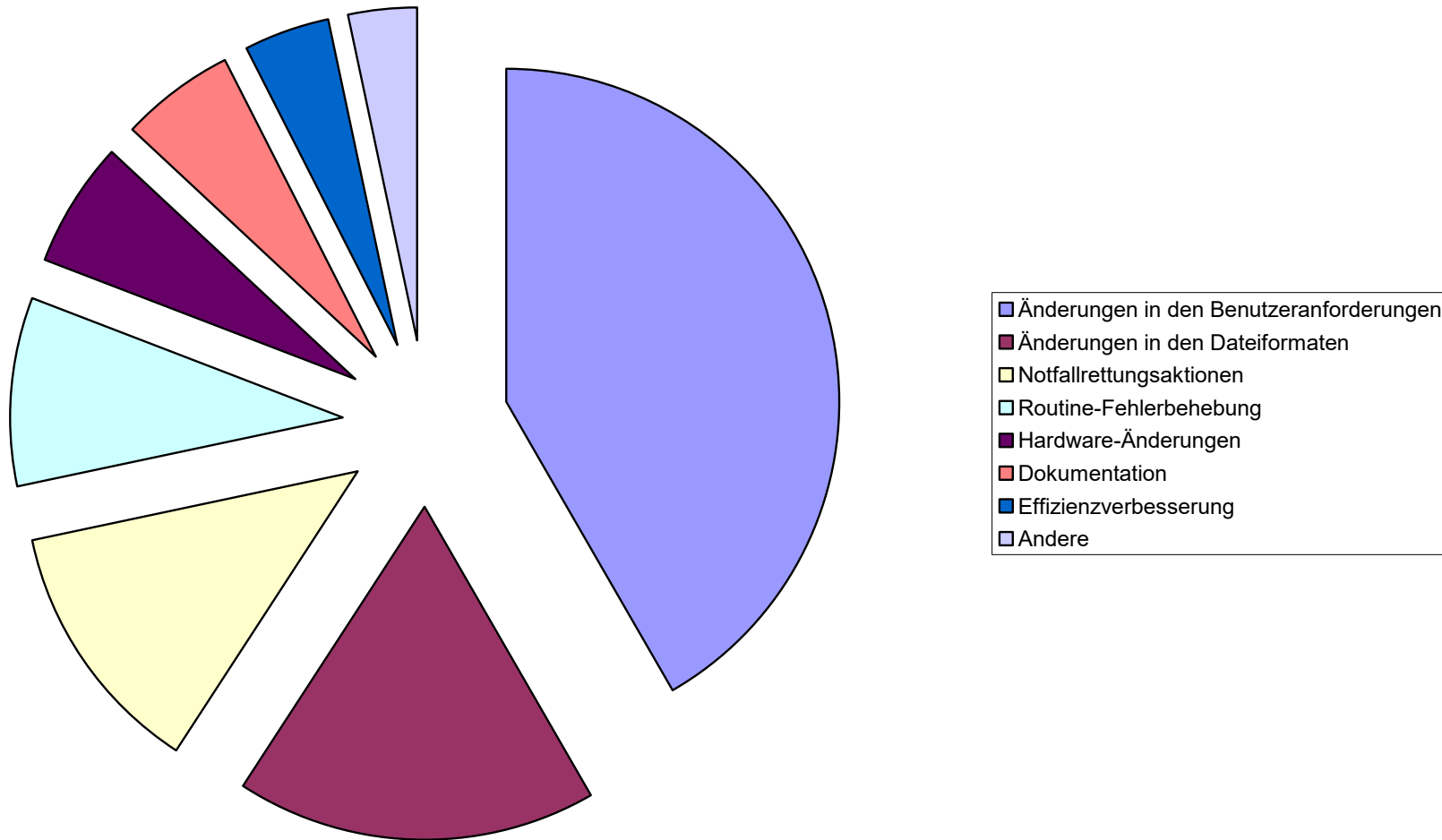
# 1992: Desktop von Windows 3.11

- GUIs sind mit funktionsorientierten Ansatz kaum zu realisieren

The screenshot shows the Windows 3.11 desktop environment. Four windows are visible, each with a corresponding metadata box:

- Window 1 (Clock):**
  - ID: 2
  - Titel: Clock
  - xPos: 0
  - yPos: 0
  - displayMode: resizable
  - inputFocus: false
- Window 2 (Program Manager):**
  - ID: 1
  - Titel: Program Manager
  - xPos: 100
  - yPos: 5
  - displayMode: resizable
  - inputFocus: false
- Window 3 (File Manager):**
  - ID: 5
  - Titel: File Manager
  - xPos: 10
  - yPos: 240
  - displayMode: resizable
  - inputFocus: false
- Window 4 (Minesweeper):**
  - ID: 3
  - Titel: Minesweeper
  - xPos: 500
  - yPos: 240
  - displayMode: resizable
  - inputFocus: true

- Softwarewartung machte Ende der 1980er ca. 70% der Gesamtkosten eines SW-Projekt aus





- Der Begriff der „**Softwarekrise**“ entsteht bereits in den 60ern
- Durch besonders spektakuläre Fälle erreichte dieser Begriff Ende der 70er eine breite Öffentlichkeit:
  - 1979 : **Studie zu Softwareprojekten** (USA), inges. 7 Mio USD
    - 75% der Ergebnisse nie eingesetzt
    - 19% der Ergebnisse stark überarbeitet
    - 6% benutzbar
  - 1981 : **US Air Force** Command & Control Software überschreitet Kostenvoranschlag fast um den Faktor 10 (3,2 Mio USD)
  - 1984 : **Überschwemmungskatastrophe** im französischen Tarntal, weil Steuercomputer nach Überlaufwarnung Schleusen öffnet
  - 1990 : Ausfall des **Telefonnetzes** in den USA
  - 1996 : Absturz der **Ariane 5** wegen eines Software-Fehlers

# Explosion der Ariane 5 am 4. Juni 1996

## Ada-Programm des Trägheits-N... (Ausschnitt):

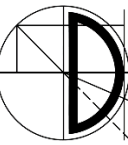
```
...  
declare  
  vertical_veloc_sensor: float;  
  horizontal_veloc_sensor: float;  
  vertical_veloc_bias: integer;  
  horizontal_veloc_bias: integer;  
...  
begin  
  declare  
    pragma suppress(numeric_error, horizontal_veloc_b  
  begin  
    sensor_get(vertical_veloc_sensor);  
    sensor_get(horizontal_veloc_sensor);  
    vertical_veloc_bias := integer(vertical_veloc_sensor);
```



Entwicklungszeit: ca. 10 Jahre  
Entwicklungskosten: ca. 6 Mrd €  
Finanzieller Schaden: ca. 1 Mrd €  
Der größte Teil der Software war  
von der Ariane 4 übernommen  
worden.

Quelle: <http://www4.in.tum.de/lehre/seminare/ps/WS0203/desaster/Riedl-Arianne5-Ausarbeitung-27-11-02.pdf>

# Ausfall des Telefonnetzes in den USA 1990



```
switch expression {  
    ...  
    case (value):  
        if (logical) {  
            <statements>  
            break;  
        } else {  
            <statements>  
        }  
        <statements>  
    ...  
}
```

Dauer: 9 Stunden

Folge: 70 Mio (von 134 Mio) Ferngespräche nicht vermittelt

Finanzieller Schaden: ca. 500 Mio €

Auslöser: Kettenreaktion ausgelöst durch eine Schaltzentrale in Manhattan

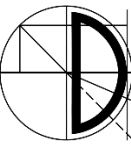
Ursache: Software Update mit falsch eingesetztem break-Befehl;  
dieses Update war zuvor auf alle Schaltzentralen verteilt worden



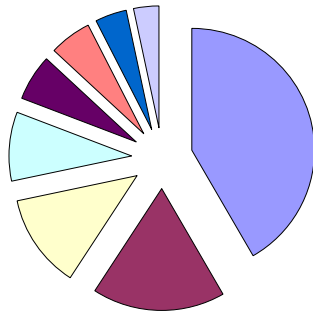
Quelle: <http://formal.iti.kit.edu/~beckert/teaching/Seminar-Softwarefehler-SS03/Ausarbeitungen/sesterhenn.pdf> und  
<http://formal.iti.kit.edu/~beckert/teaching/Seminar-Softwarefehler-SS03/Folien/sesterhenn.pdf>



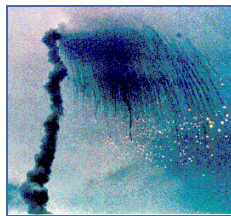
# Die Treiber für die Objektorientierung (Ende der 1980er Jahre)



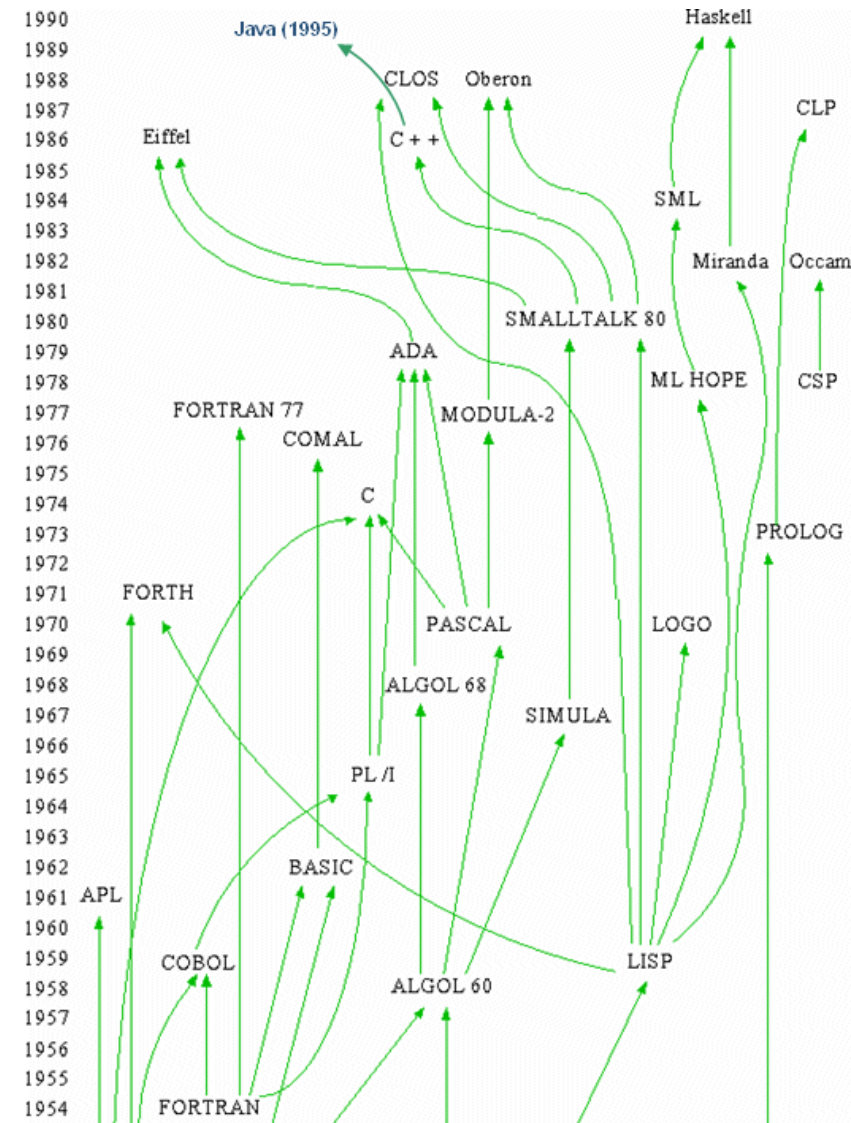
**Grafische Benutzer-Oberflächen** für den neuen PC-Massen-Markt (Programm-Komplexität vervielfacht sich)!

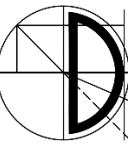


Die **Kosten für die Wartung** der Software sind zu hoch, Anpassungen dauern zu lange und kosten zu viel!

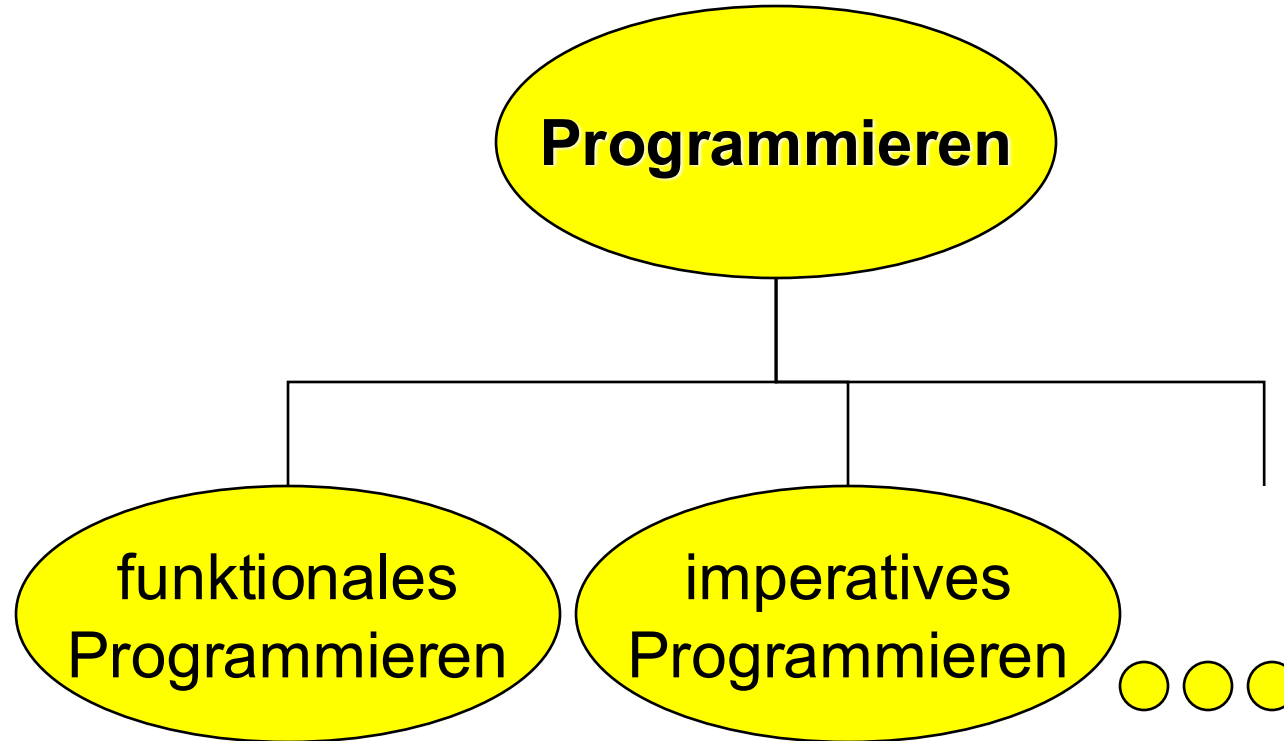


**Softwarekrise** führt zu immer größeren und imageschädigenden Unglücken!

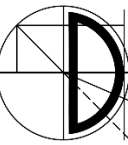




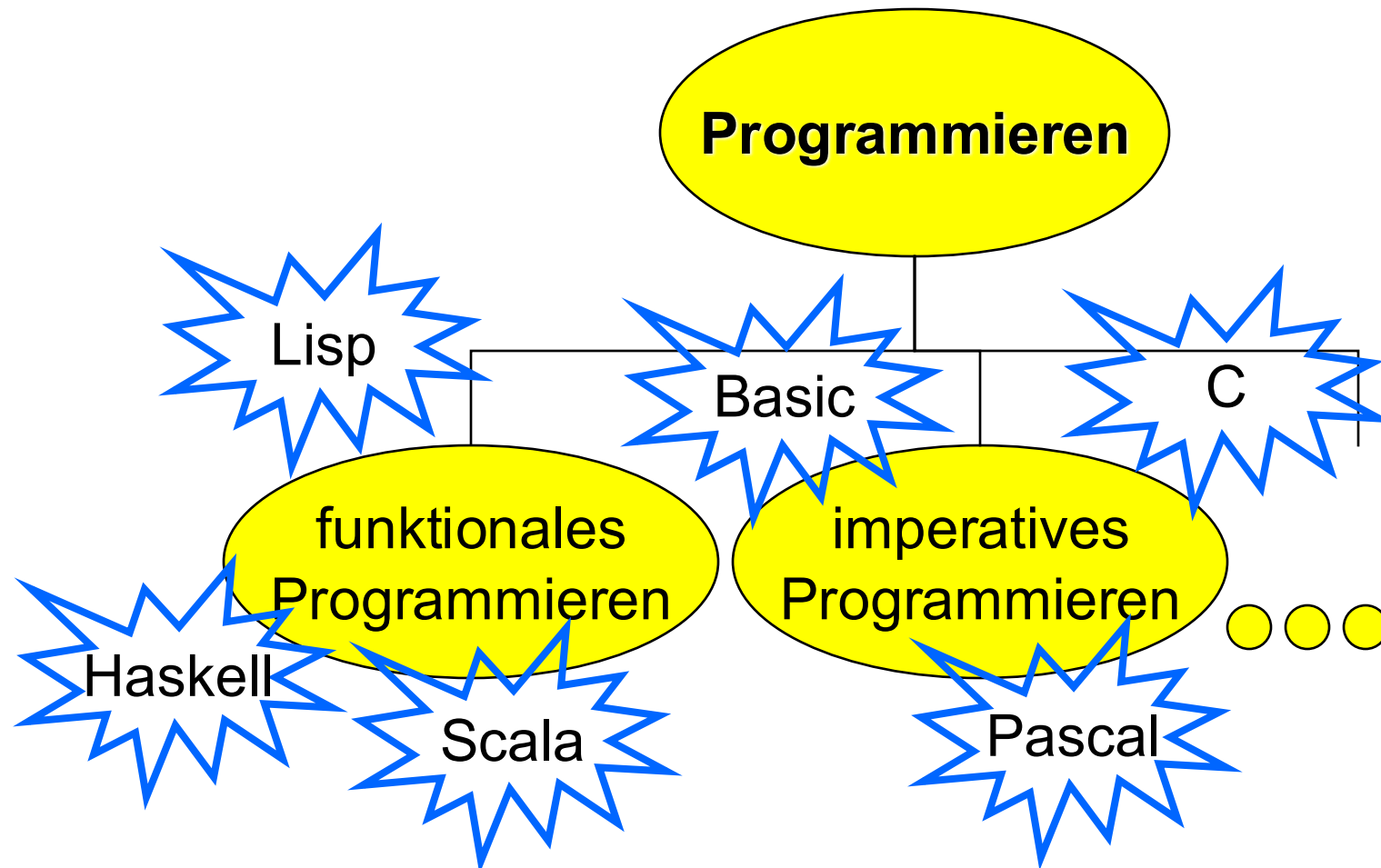
- Es gibt verschiedene Philosophien, wie Programme auf dem Rechner verfasst werden sollten, sogenannte **Paradigmen**

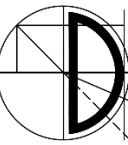




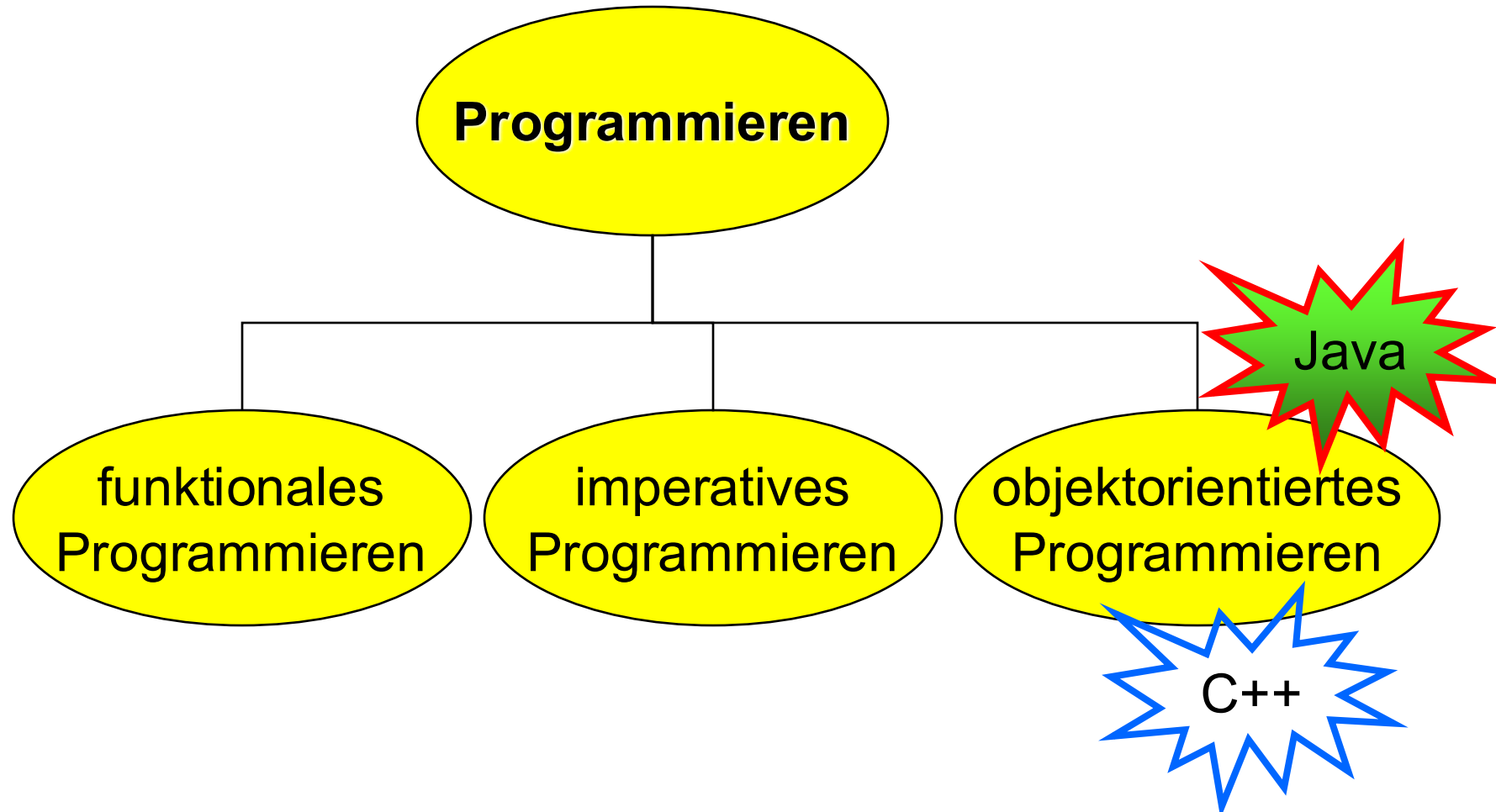


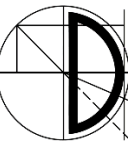
- In der Entwicklungsgeschichte der Programmiersprachen haben sich zu den verschiedenen Paradigmen unterschiedliche Sprachen entwickelt...



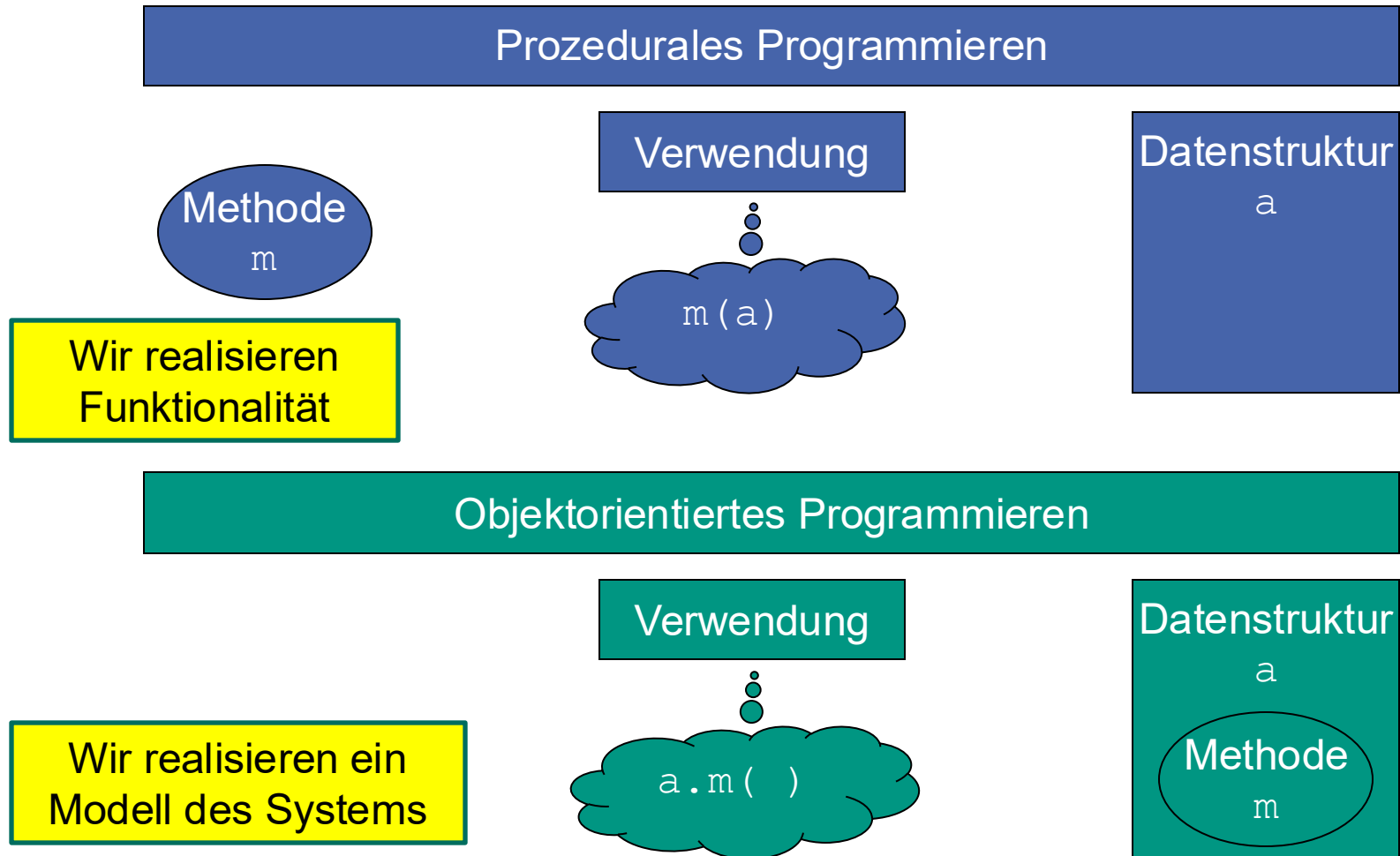


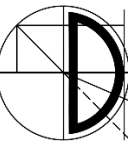
- Der Programmiersprache Java liegt der Ansatz des **objektorientierten Programmierens** zugrunde





- Statt Methoden dominieren Datenstrukturen





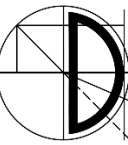
- Verschiedene Formen des objektorientierten Ansatzes
  - **Objektbasierte Programmierung** ist die Entwicklung von Softwaresystemen als Sammlung von Implementierungen abstrakter Datentypen

Struktur = Vererbungsbaum

- **Objektorientierte Programmierung** ist die Entwicklung von Softwaresystemen als **strukturierte** Sammlung von Implementierungen Abstrakter Datentypen

*"Object-oriented software construction is the building of software systems as structured collections of possibly partial abstract data type implementations."*

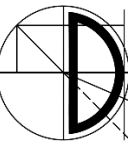
Bertrand Meyer, Object-Oriented Software Construction, 2nd Ed., Prentice Hall PTR, 2009, page 147



- Objektorientierte Programmierung geht von dem Grundsatz aus, dass ein wie auch immer geartetes Programm meist einen Teil der Welt widerspiegelt, in der wir leben
- Wenn wir also ein **Modell** dieser Welt auf dem Computer erzeugen wollen, sollten wir die einzelnen **Objekte**, mit denen wir im Leben konfrontiert werden, auf dem Computer abbilden können

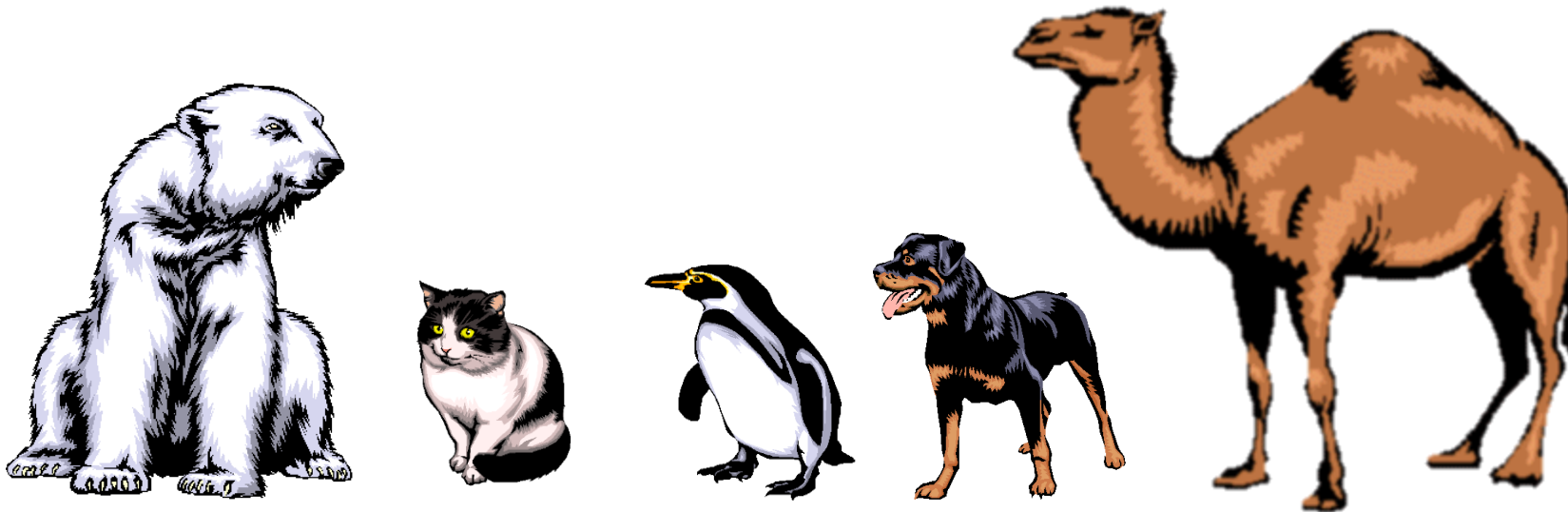


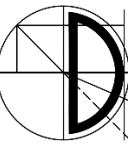




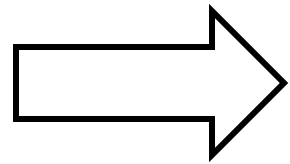
## Beispiel

Wir wollen auf unserem Computer  
verschiedene Tiere „simulieren“

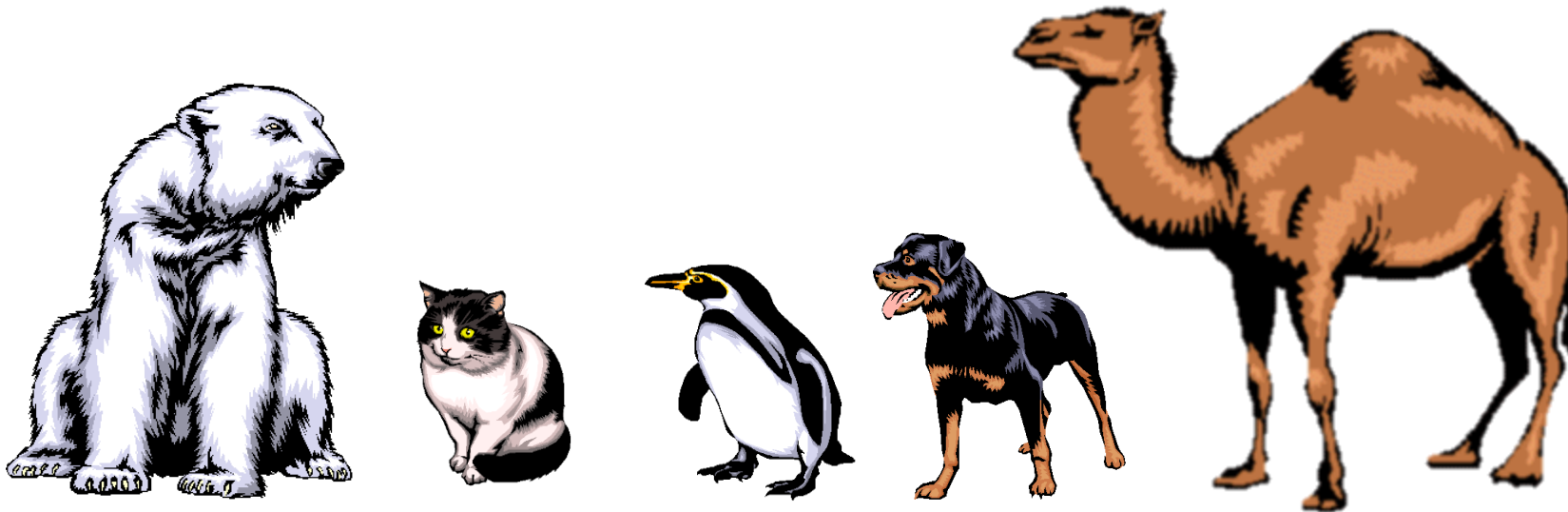


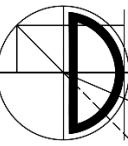


- Würden wir (wie bisher) imperativ programmieren, so müssten wir in Java eine Unmenge von Daten und Methoden erstellen, um das Verhalten der einzelnen Tiere abzubilden

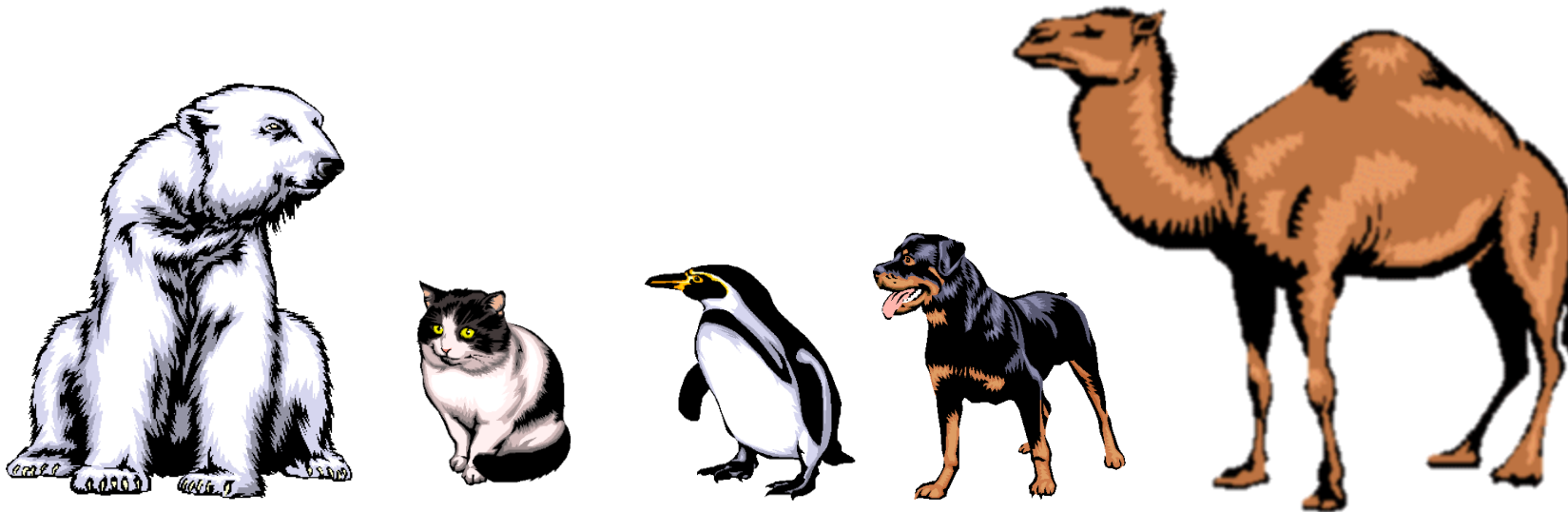


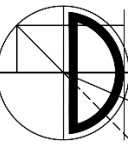
Unser Programm würde  
unübersichtlich und kompliziert!





- Mit Hilfe des objektorientierten Ansatzes umgehen wir dieses Problem, indem wir die auftretenden Objekttypen durch verschiedene **Klassen** (abstrakte Datentypen, Schlüsselwort **class**) modellieren





- Mit Hilfe des objektorientierten Ansatzes umgehen wir dieses Problem, indem wir die auftretenden Objekttypen durch verschiedene **Klassen** (abstrakte Datentypen, Schlüsselwort **class**) modellieren

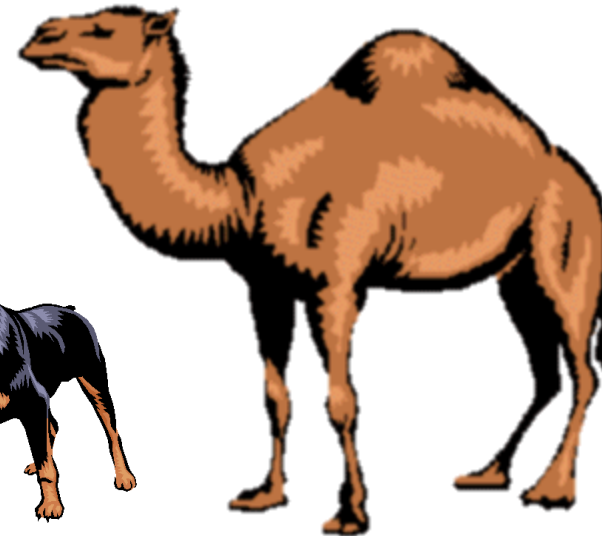
Eisbaer

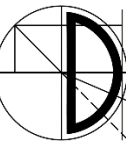
Katze

Pinguin

Hund

Dromedar





- Jede dieser Klassen repräsentiert die gesammelten Eigenschaften eines Objekttyps und stellt somit sozusagen dessen „Bauplan“ (abstrakter Datentyp) dar

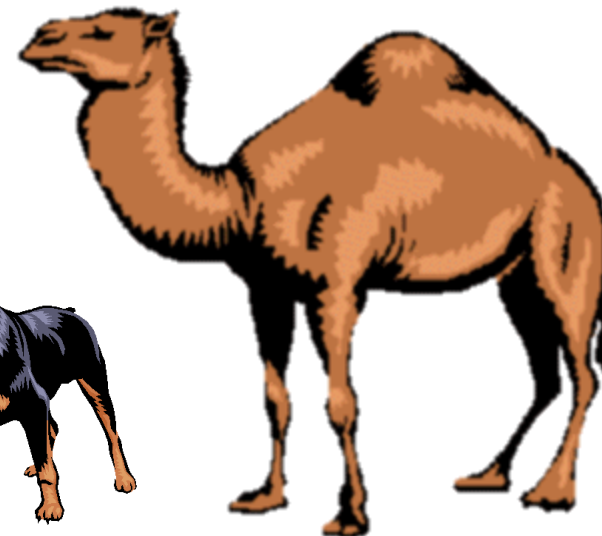
Eisbaer

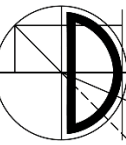
Katze

Pinguin

Hund

Dromedar

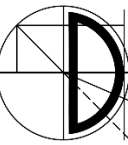




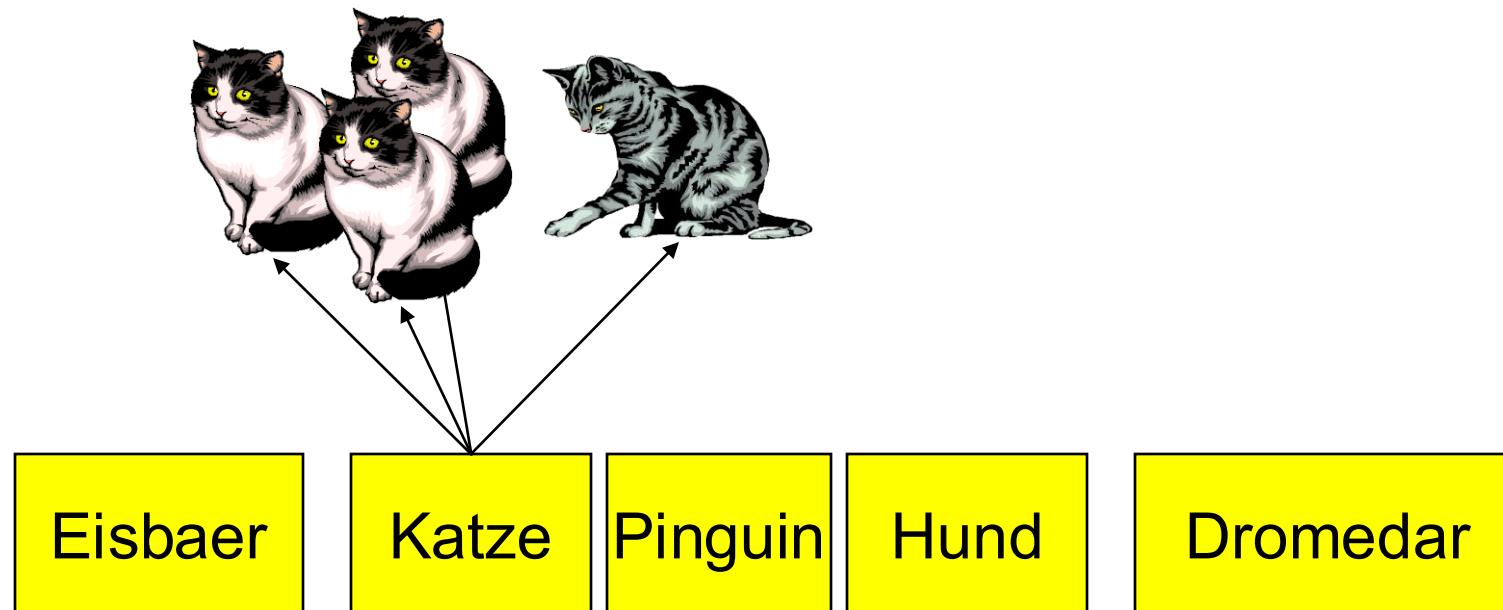
- Wollen wir nun in unserem Programm etwa ein Exemplar der Gattung „Katze“ verwenden, so können wir unsere Klassenbeschreibung benutzen, um aus ihr das gewünschte **Objekt** zu **erzeugen**
- Man bezeichnet diesen Vorgang als **Instantiierung**

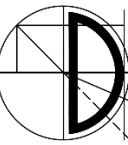




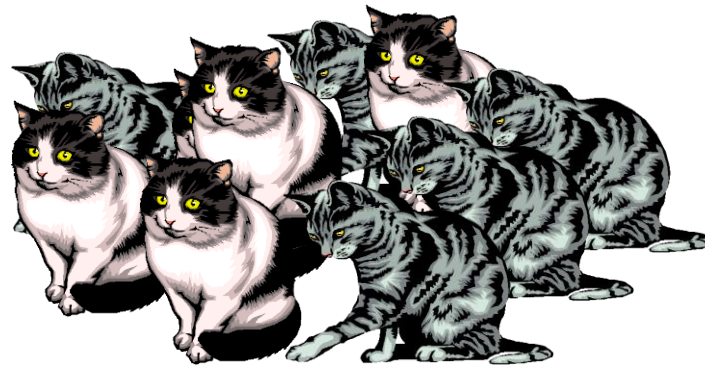


- Natürlich ist es auch möglich, anhand **einer** Klassenbeschreibung **mehrere** Objekte zu erzeugen
- Wie bei verschiedenen Häusern, die nach dem selben Bauplan gebaut wurden, ist jedes Objekt individuell verschieden und kann seine eigenen Besonderheiten haben





- Egal jedoch, wie viele **Instanzen** unserer Klasse wir bilden - es handelt sich stets um ein und denselben Bauplan, welchen wir beliebig oft als Vorlage verwenden können



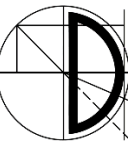
Eisbaer

Katze

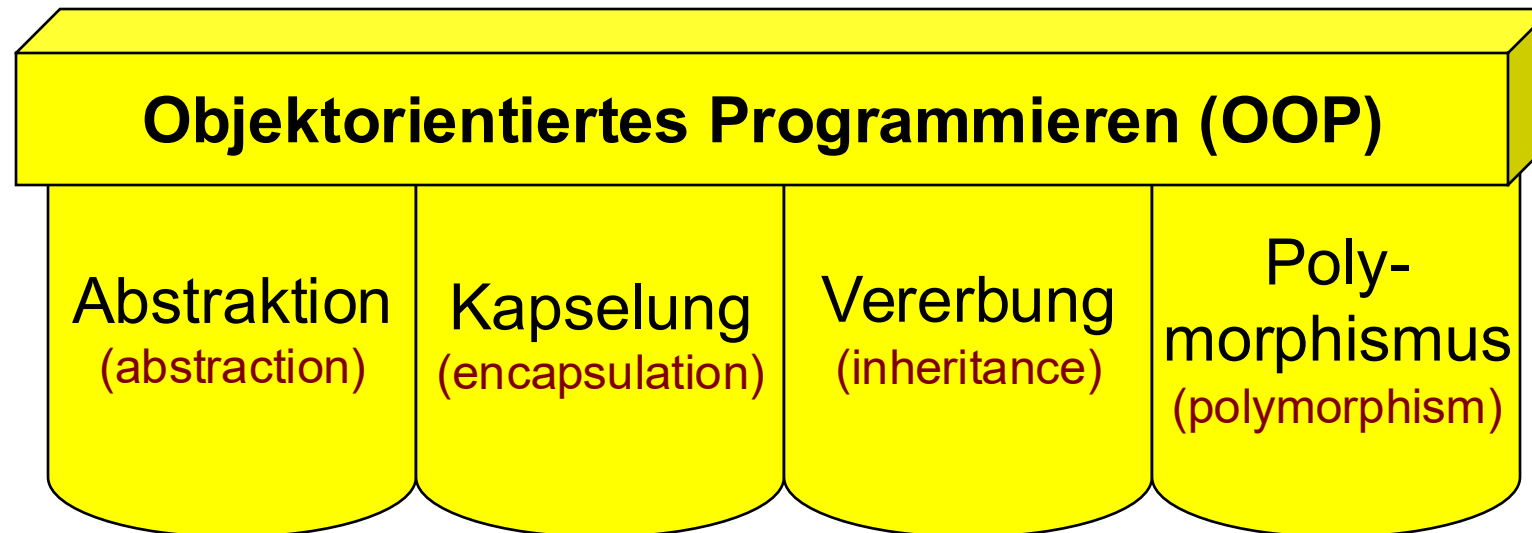
Pinguin

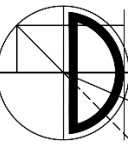
Hund

Dromedar



- Die objektorientierte Philosophie stützt sich allgemein auf vier grundlegende Prinzipien:

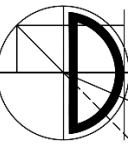




ist das Zusammenfassen mehrerer Klassen mit gemeinsamen Merkmalen zu einer allgemeineren Oberklasse

Ziel der **Abstraktion** ist es,

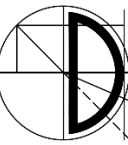
- eine **Hierarchie** zu erzeugen, in der Objekte wie in der wahren Welt miteinander in Beziehung stehen,
- gemeinsame **Eigenschaften** verschiedener Klassen **einheitlich** zu modellieren und somit
- **Entwicklungszeit** zu sparen und
- eine Vielzahl von **Fehlerquellen** auszuschalten



## Beispiel

Wir wollen unsere fünf Tierarten in verschiedene Kategorien unterteilen





## Beispiel

Wir wollen unsere fünf Tierarten in verschiedene Kategorien unterteilen

Wir wollen als erstes Kriterium unterscheiden, ob es sich bei den Tieren um **Säugetiere** oder **Vögel** handelt

Eisbaer

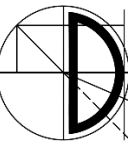
Katze

Pinguin

Hund

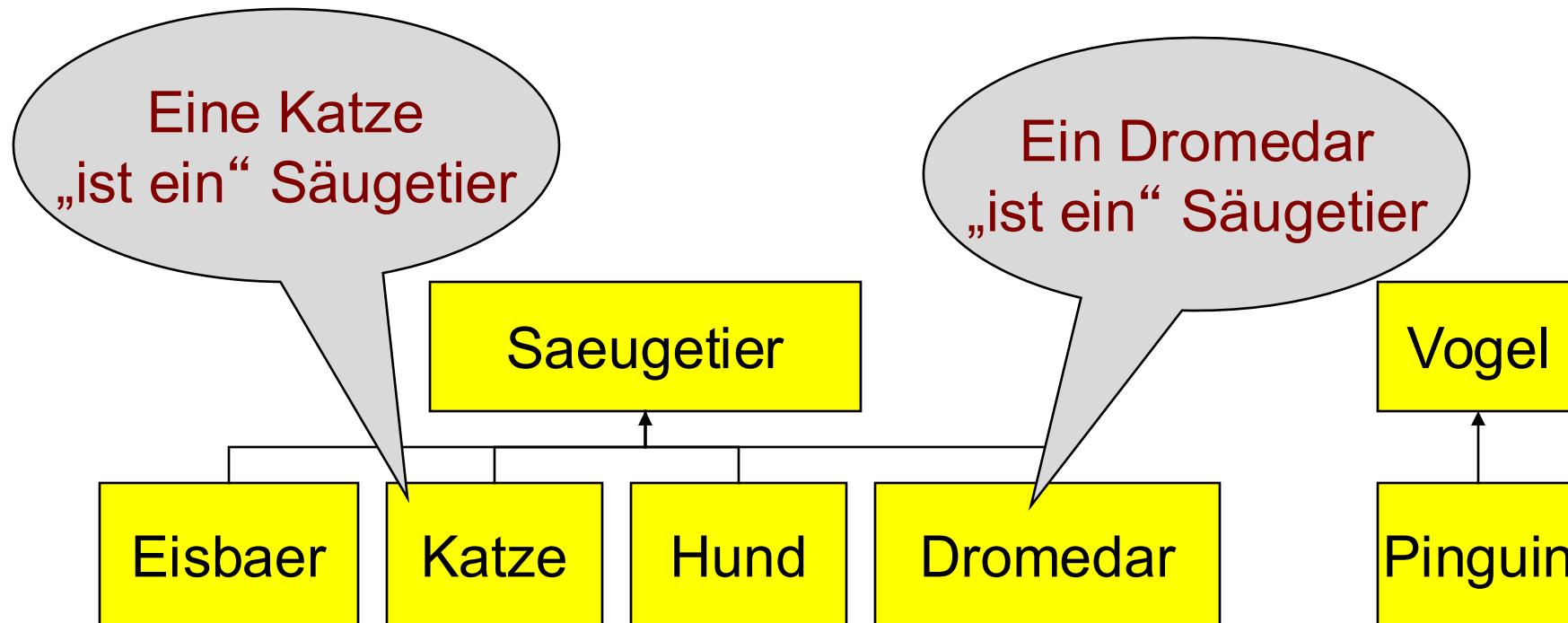
Dromedar

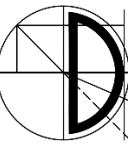




## Beispiel

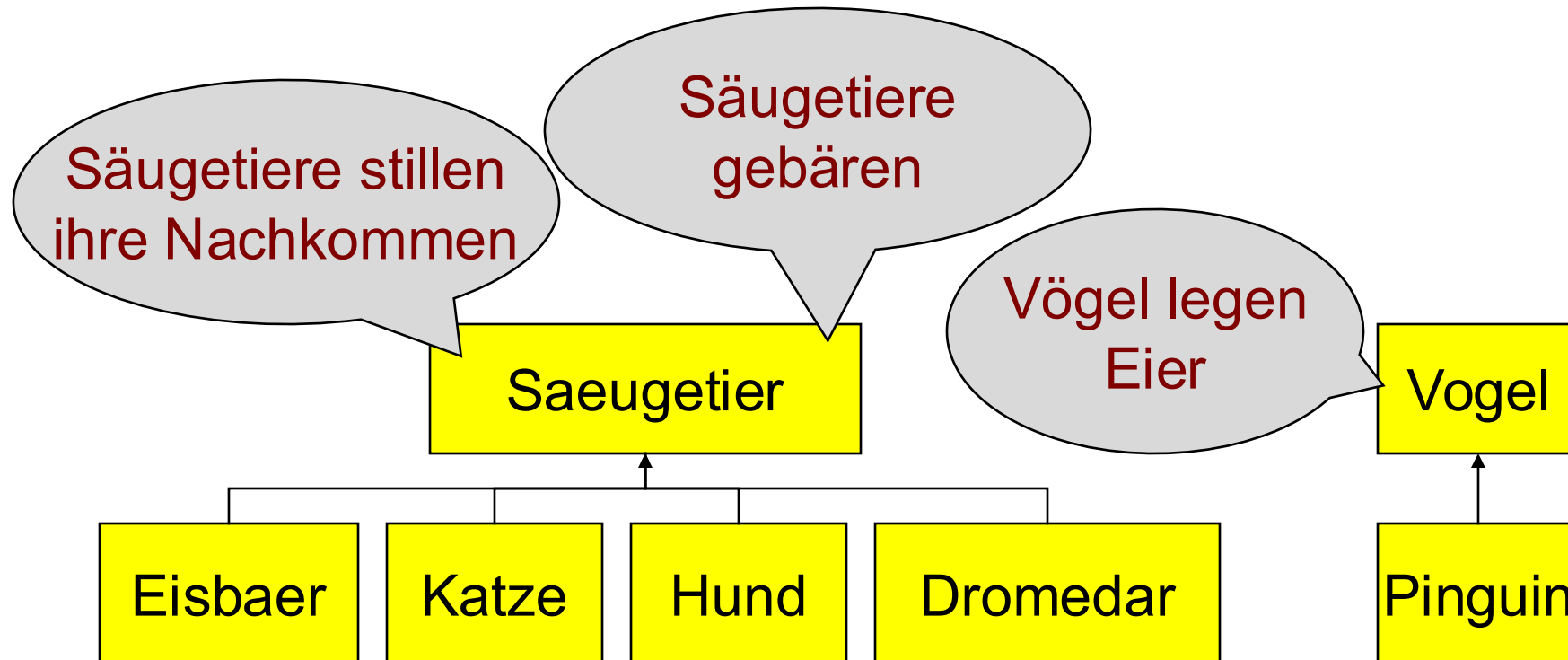
Bei der hier angegebenen Abstraktion (sie wird auch als **Generalisierung** bezeichnet), handelt es sich um eine „ist-ein“-Beziehung

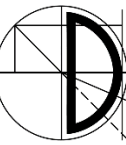




## Beispiel

Sämtliche Tiere, für die diese „ist-ein“-Beziehung gilt, haben gewisse Eigenschaften gemeinsam

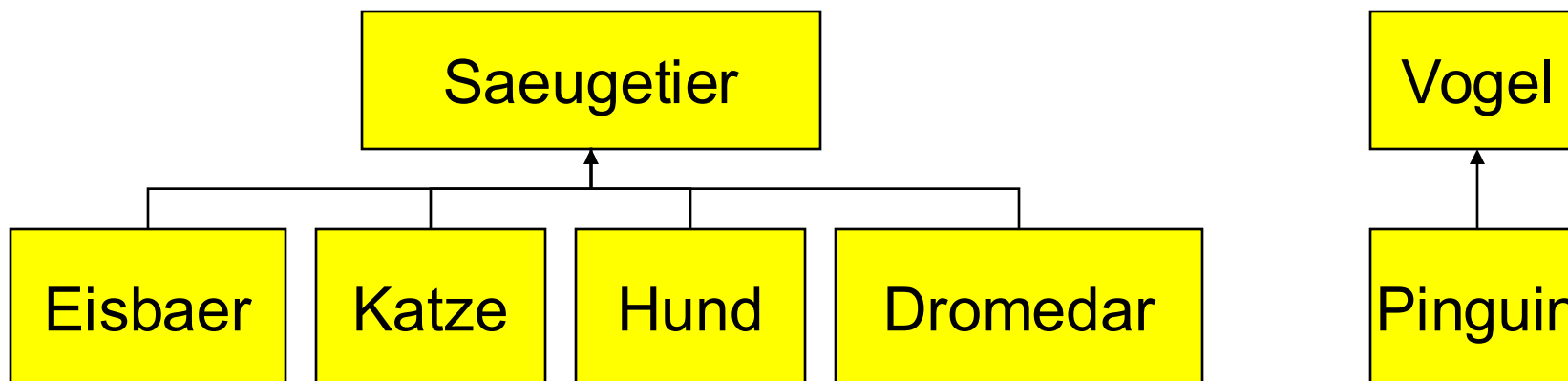


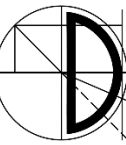


## Beispiel

Sämtliche Tiere, für die diese „ist-ein“-Beziehung gilt, haben gewisse Eigenschaften gemeinsam

Anstatt also etwa den Vorgang des „Milchgebens“ für Eisbär, Katze, Hund und Dromedar einzeln zu modellieren, wird der geschickte Programmierer dies lediglich für die **Superklasse** „Saeugetier“ tun



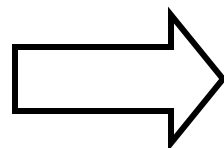


## Beispiel

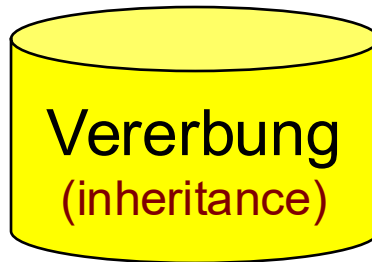
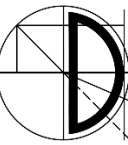
Sämtliche Tiere, für die diese „ist-ein“-Beziehung gilt, haben gewisse Eigenschaften gemeinsam

Anstatt also etwa den Vorgang des „Milchgebens“ für Eisbär, Katze, Hund und Dromedar einzeln zu modellieren, wird der geschickte Programmierer dies lediglich für die **Superklasse** „Saeugetier“ tun

Hierdurch erhalten auch alle **Subklassen** (das sind die Klassen, die zur Superklasse in einer „ist-ein“-Beziehung stehen) automatisch die Eigenschaft, Milch geben zu können



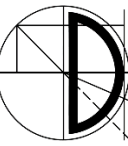
Wir bezeichnen diesen Vorgang als **Vererbung**



ist das Weiterreichen von Eigenschaften zwischen Klassen, die in einer „ist-ein“-Beziehung stehen

Ziel der **Vererbung** ist es

- gemeinsame **Eigenschaften** verwandter Klassen **einheitlich** zu modellieren,
- möglichst viel **Programmcode wiederverwenden** zu können und somit
- **Entwicklungszeit** zu sparen und
- eine Vielzahl von **Fehlerquellen** auszuschalten

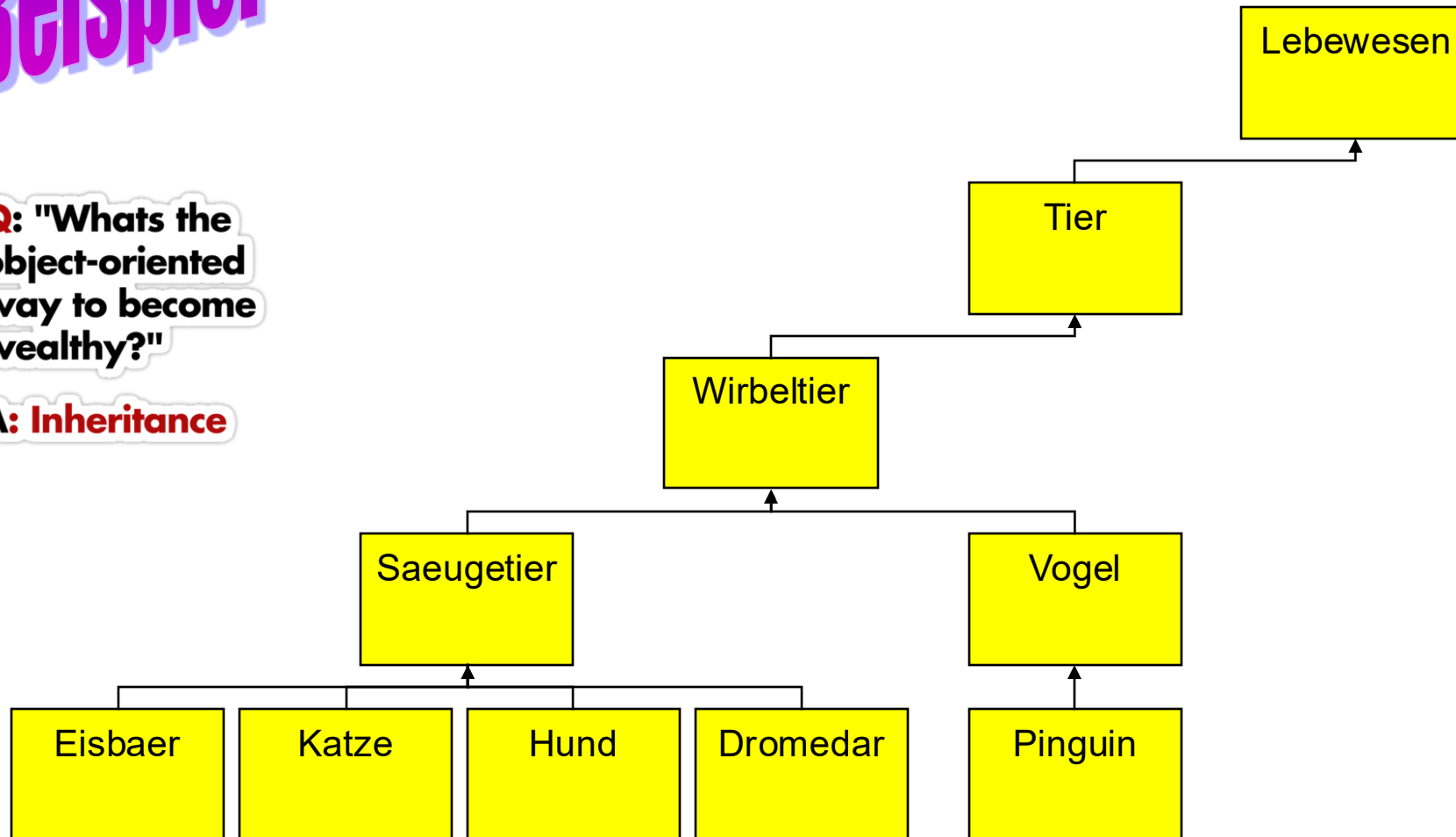


## Beispiel

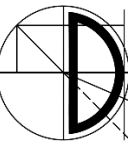
Betrachten wir eine „erweiterte Version“ unserer Tierhierarchie:

**Q:** "Whats the object-oriented way to become wealthy?"

**A:** Inheritance

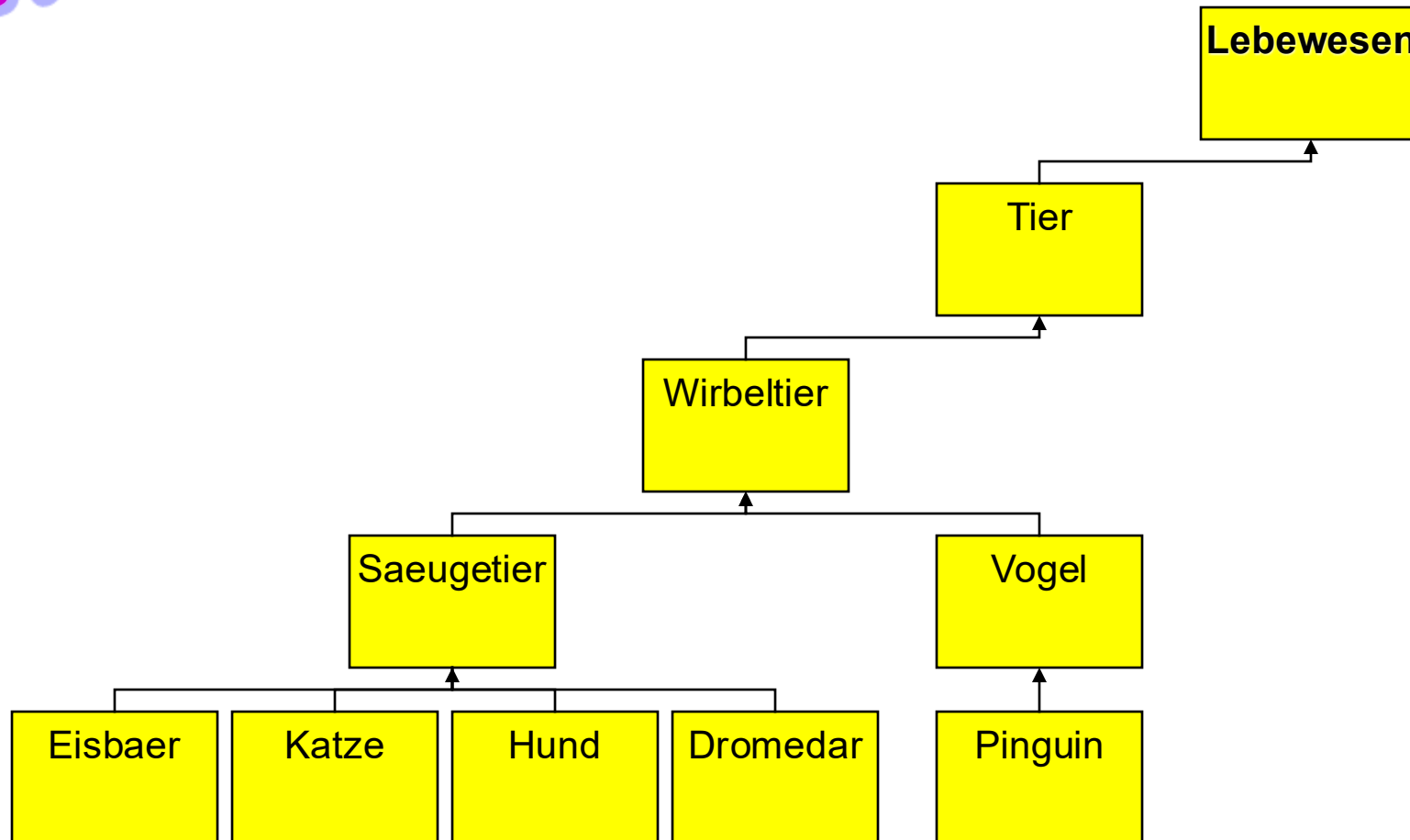


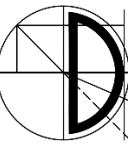




## Beispiel

Jede Klasse in diesem Schaubild „ist ein“  
Lebewesen und soll deshalb mit diesem gewisse  
Eigenschaften teilen

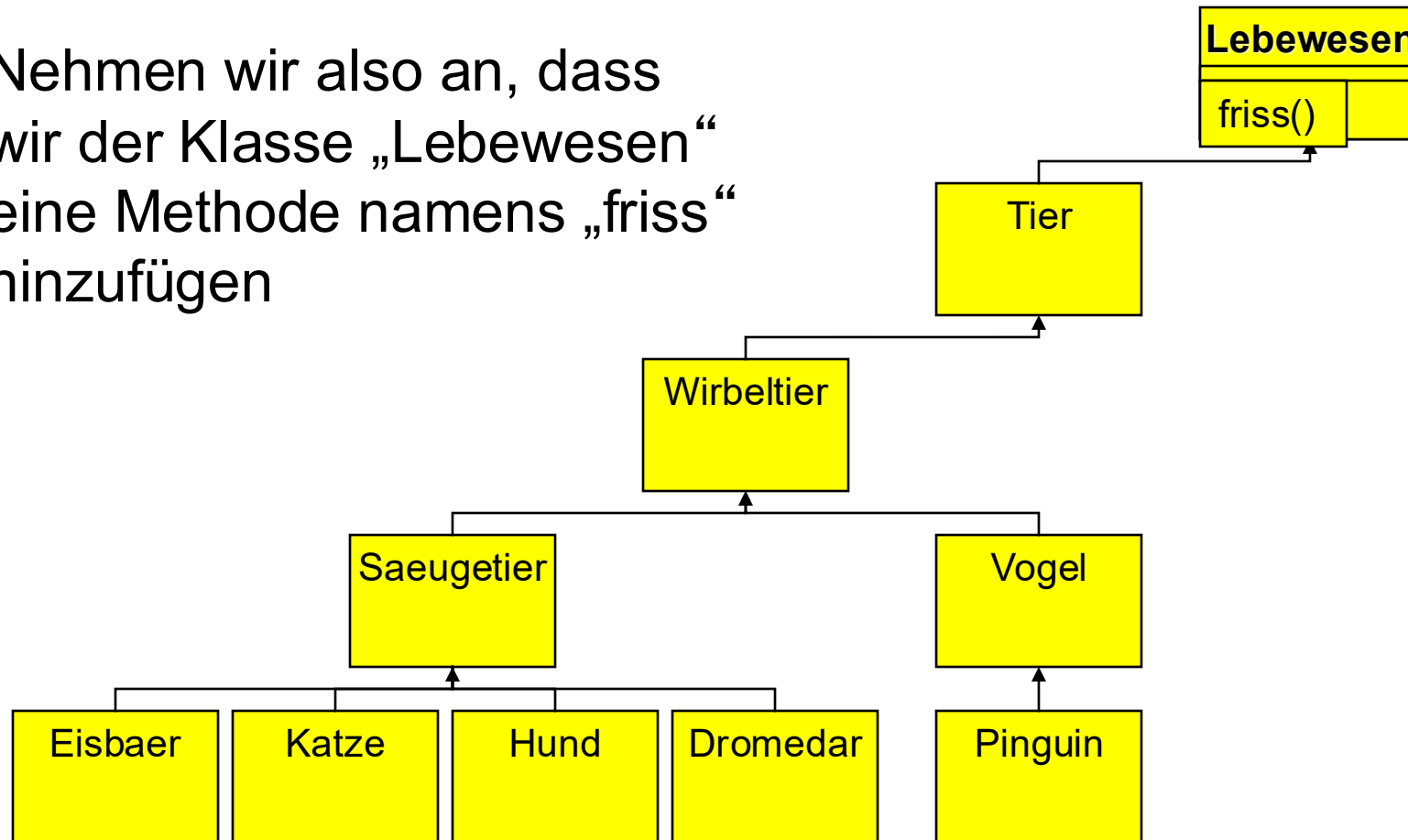


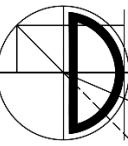


## Beispiel

Jedes Lebewesen muss beispielsweise essen

Nehmen wir also an, dass wir der Klasse „Lebewesen“ eine Methode namens „friss“ hinzufügen

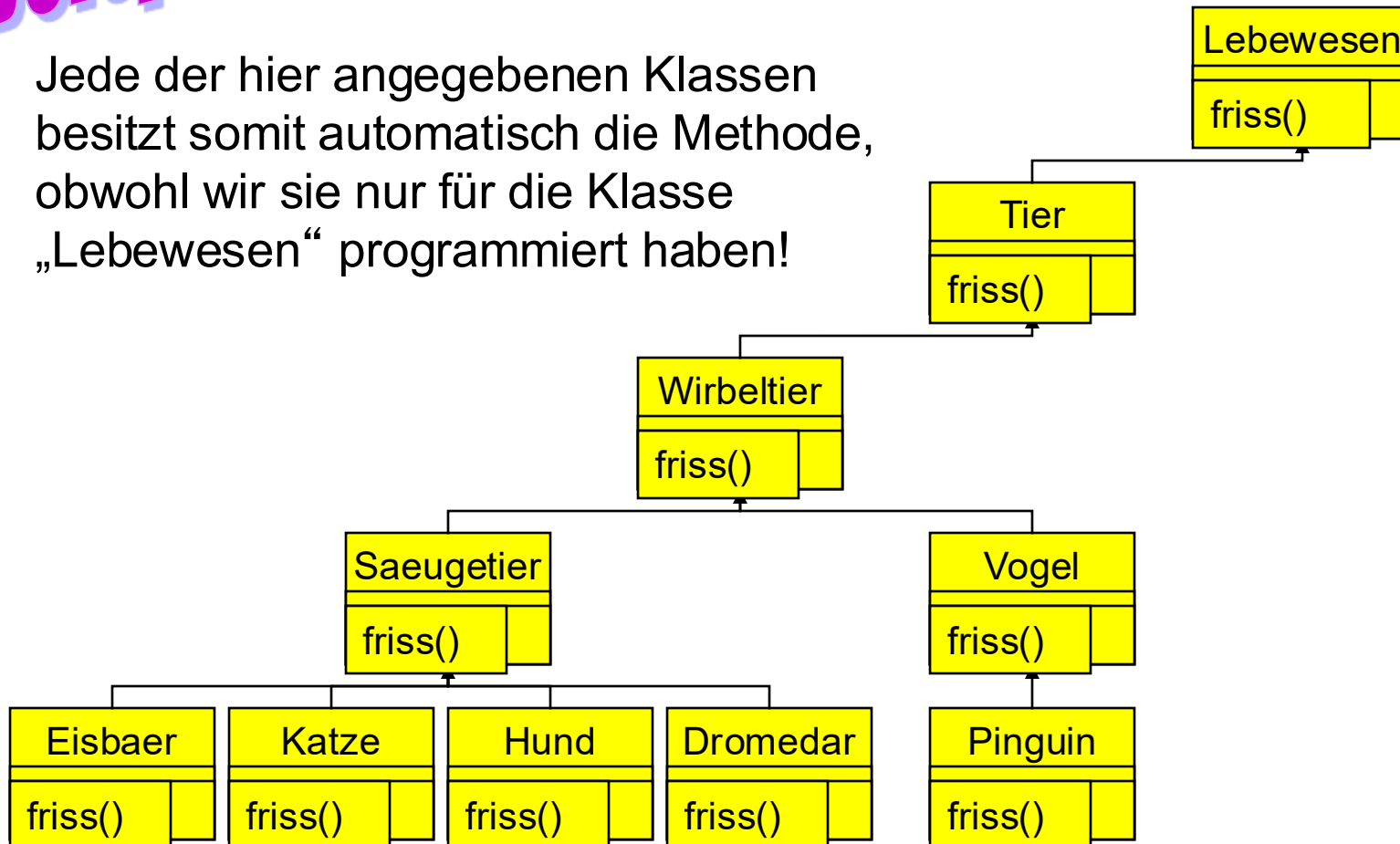


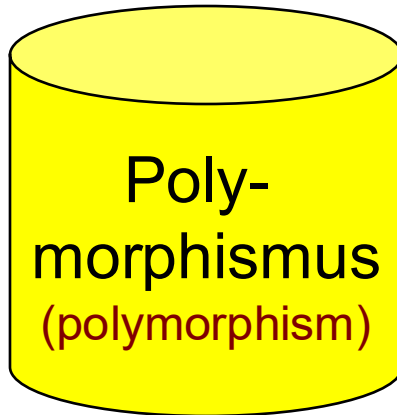
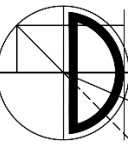


## Beispiel

Dann **vererbt** die Klasse diese Methode **automatisch** an alle ihre Subklassen

Jede der hier angegebenen Klassen besitzt somit automatisch die Methode, obwohl wir sie nur für die Klasse „Lebewesen“ programmiert haben!





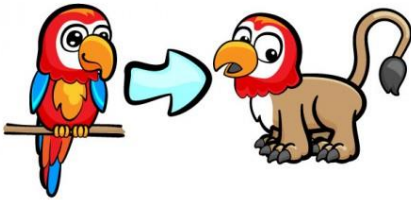
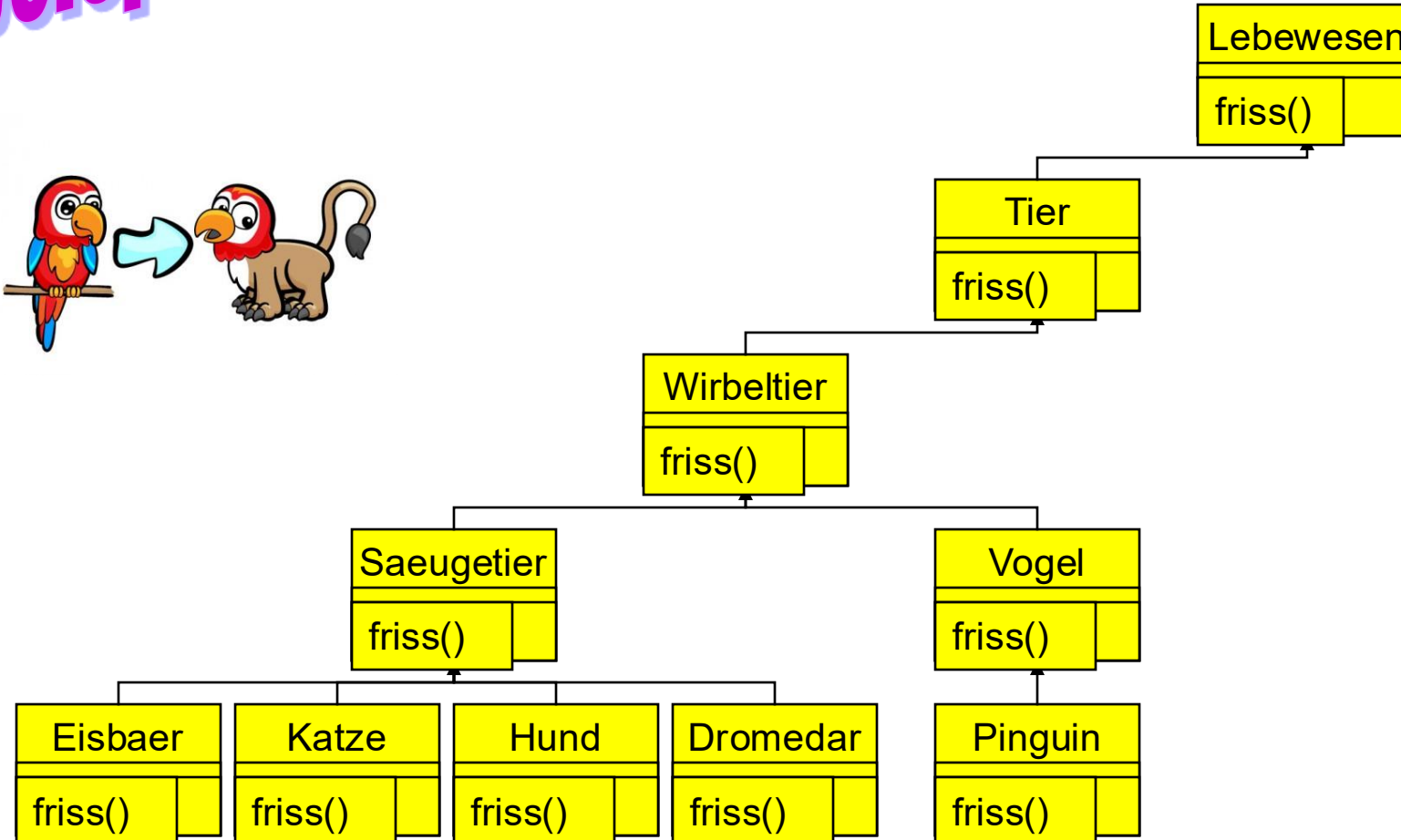
ist die Möglichkeit, Methoden innerhalb der Objekthierarchie zu **überschreiben** (override) und somit individuelles Verhalten verschiedener Klassen auf dieselbe Anweisung zu modellieren

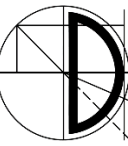
Mit Hilfe des **Polymorphismus** kann der Entwickler

- ohne Probleme **Besonderheiten** bei den verschiedenen Subklassen **berücksichtigen**,
- **einheitliche Schnittstellen** erzeugen und somit
- **Entwicklungszeit** sparen und
- eine Vielzahl von **Fehlerquellen** ausschalten

**Beispiel**

Werfen wir erneut einen Blick auf unser Tiermodell:

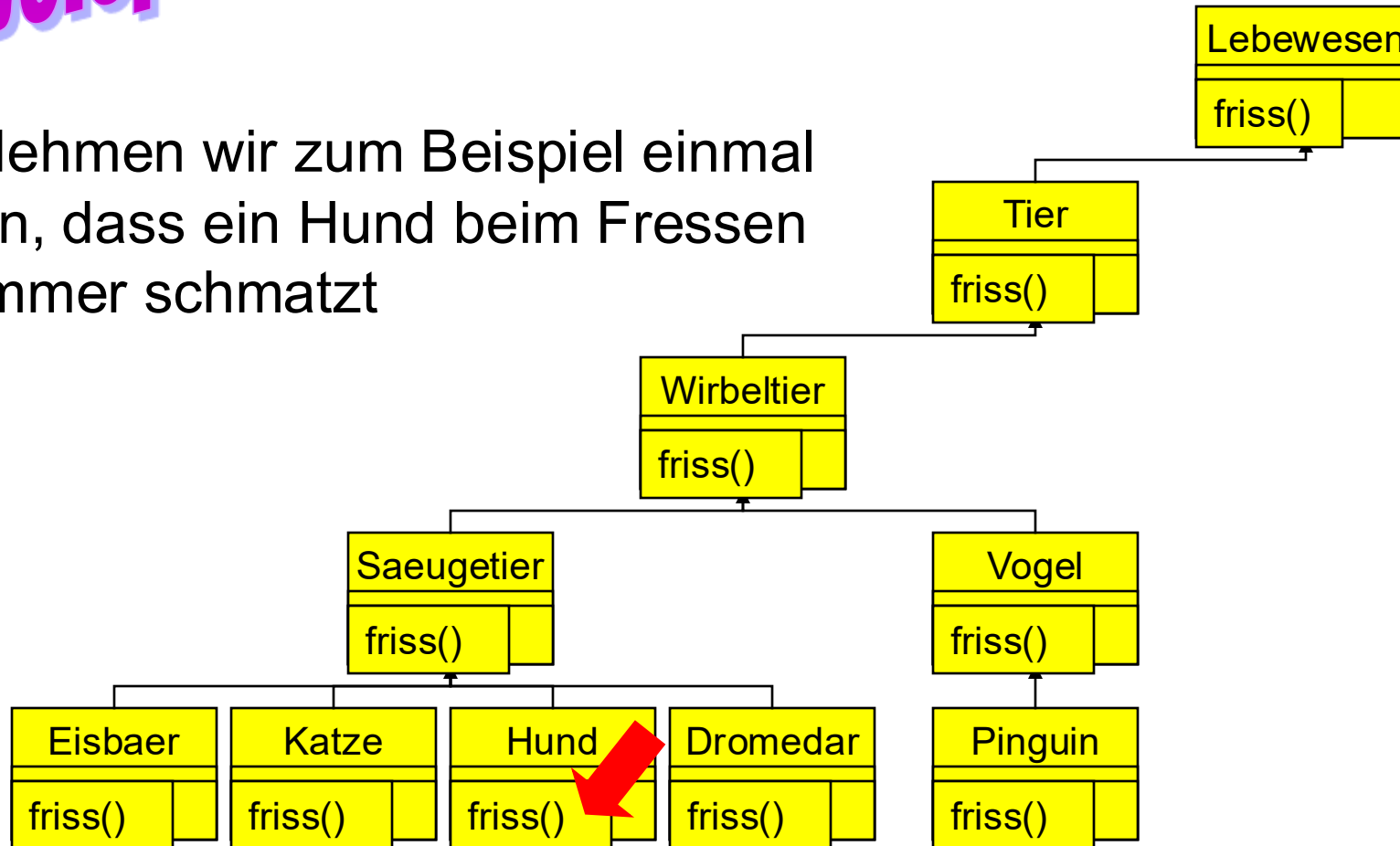


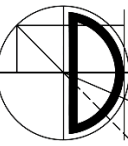


## Beispiel

Nicht jedes Lebewesen frisst in unserer Simulation auf dieselbe Weise

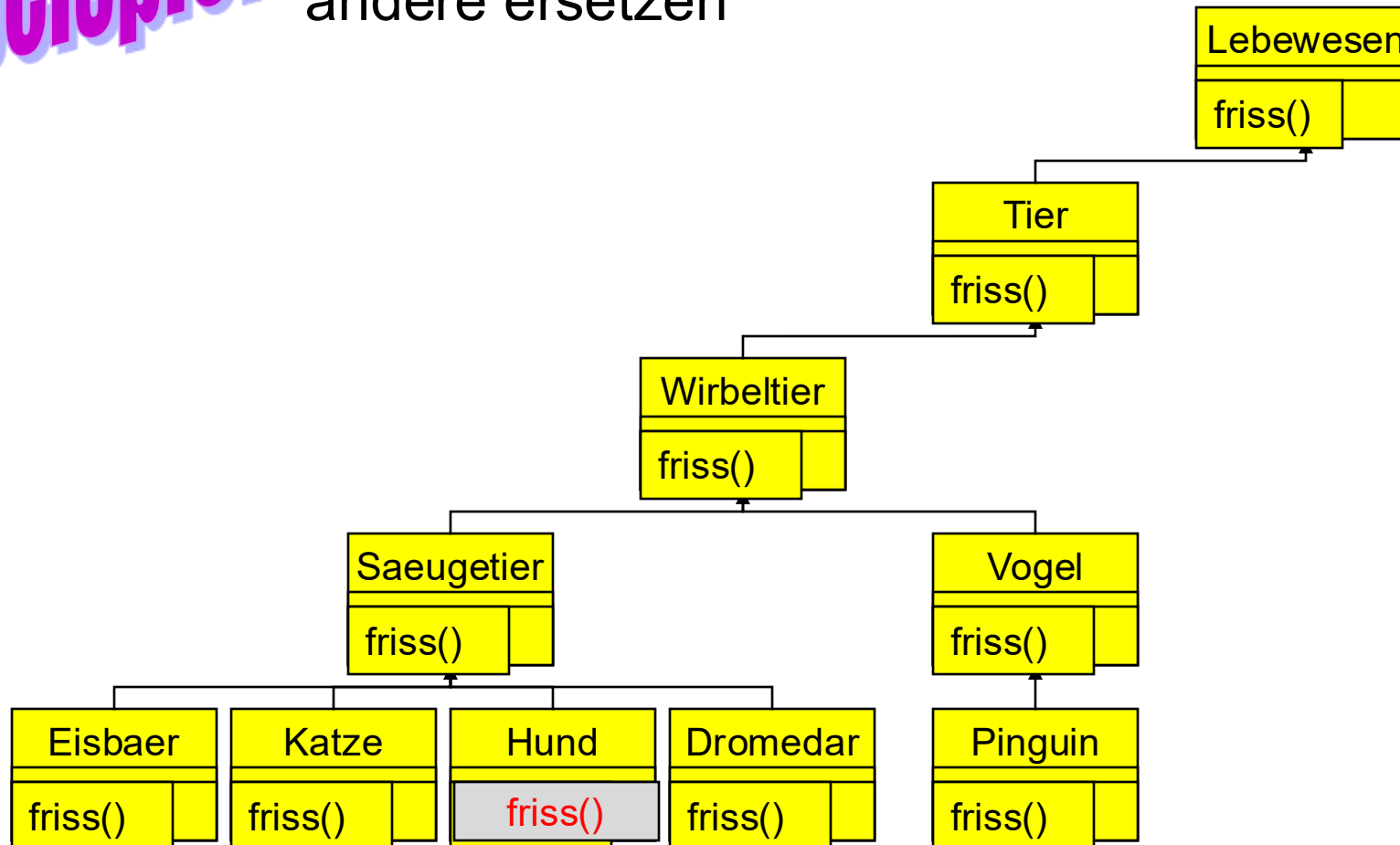
Nehmen wir zum Beispiel einmal an, dass ein Hund beim Fressen immer schmatzt



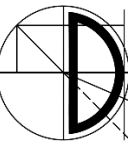


## Beispiel

In dieser Situation können wir die alte Methode „friss“ für die Klasse Hund einfach durch eine andere ersetzen



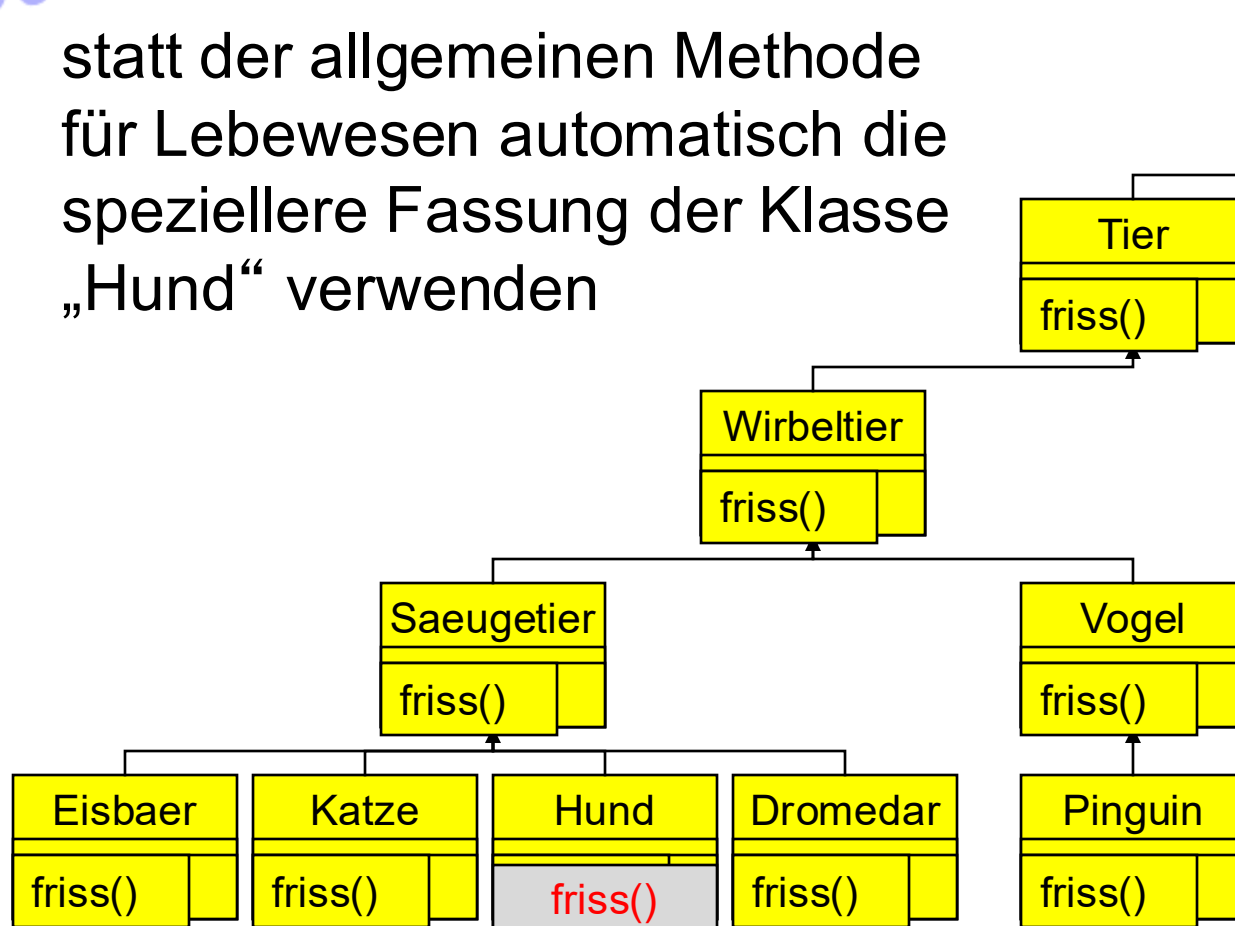




## Beispiel

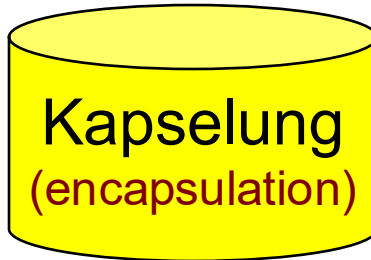
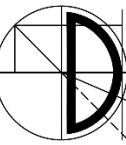
Wenn wir nun für ein Lebewesen der Gattung „Hund“ die Methode „friss“ aufrufen, wird das System

statt der allgemeinen Methode für Lebewesen automatisch die speziellere Fassung der Klasse „Hund“ verwenden



Hierzu sind vom Programmierer

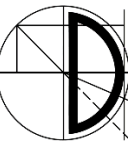
keinerlei aufwändige Fallunterscheidungen zu machen!



ist die Philosophie, **Daten und Methoden** fest an ihr zugehöriges Objekt zu **binden**

Hierbei werden Daten dem Benutzer normalerweise nicht direkt zugänglich gemacht, sondern vor direkten Zugriffen von außen geschützt (**information hiding**). Das gesamte Verhalten eines Objekts wird nur durch seine Methoden bestimmt.

Durch diese Trennung von **Schnittstelle** (der nach außen sichtbaren Methoden) und der **internen Realisierung** werden Programmierer vor Problemen bewahrt, die sich bei einer internen Umstrukturierung der Klasse ergeben.

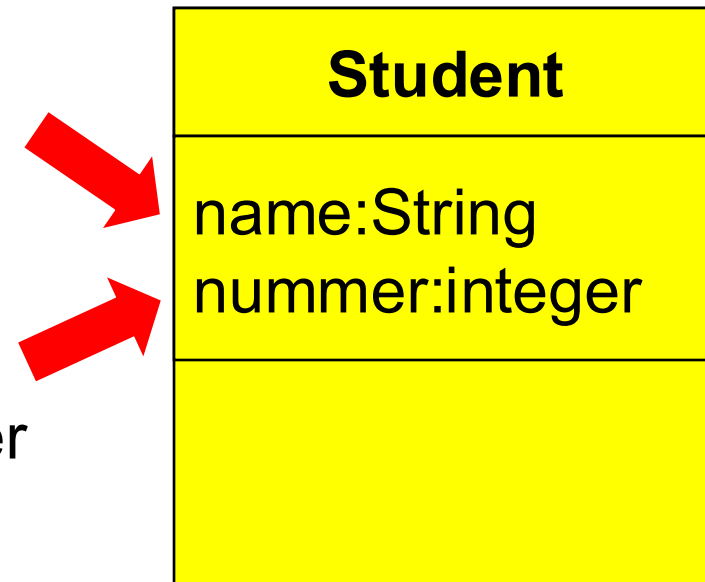


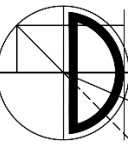
## Beispiel

Wir wollen eine Klasse „Student“ entwerfen, die Namen und Matrikelnummer eines Studenten abspeichern kann

Hierzu spendieren wir der Klasse zwei Variablen:

- in einem String namens „name“ wollen wir den Namen des Studenten ablegen und
- in einer Integer-Variable namens „nummer“ soll die Matrikelnummer gespeichert werden





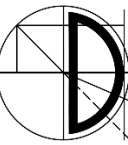
## Beispiel

Wenn wir nun aus unserem „Bauplan“ verschiedene **Instanzen** erzeugen, können wir für jede Instanz die Variablen individuell belegen

Peter:Student  
name=„Peter Petersen“  
nummer=0978532

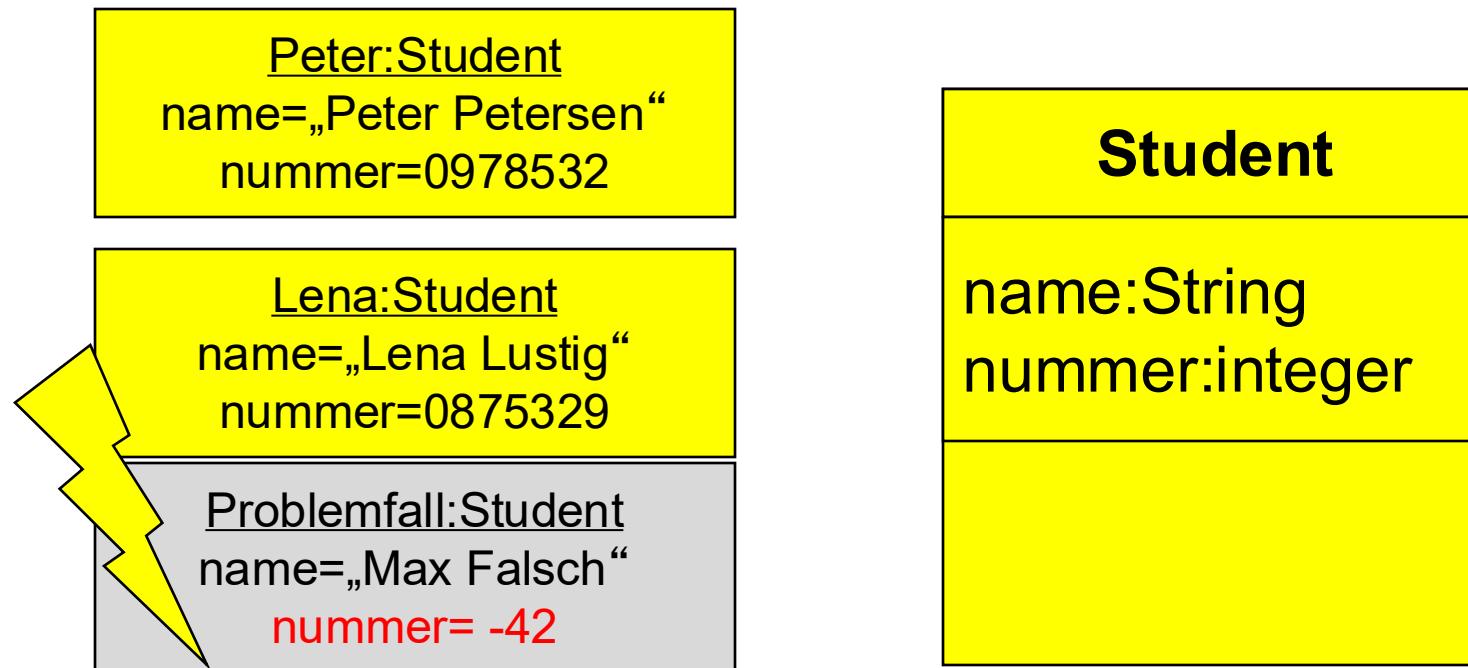
Lena:Student  
name=„Lena Lustig“  
nummer=0875329

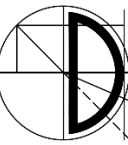
| Student                       |
|-------------------------------|
| name:String<br>nummer:integer |
|                               |



## Beispiel

Hierbei können wir jedoch nicht verhindern, dass etwa bei der Matrikelnummer eine fehlerhafte Zuweisung erfolgt





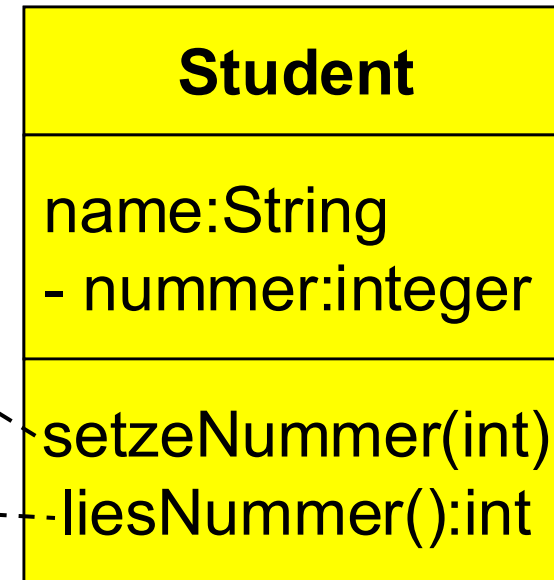
## Beispiel

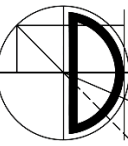
Um diese Probleme zu vermeiden, wird man die **internen Datenstrukturen** einer Klasse vor den Nutzern üblicherweise **verbergen** - in diesem Fall also etwa die Variable „nummer“

**Zugriffe** auf die entsprechenden Daten werden stattdessen **über Methoden** realisiert

Setzt die Matrikelnummer des Studenten auf den übergebenen Wert. Bei ungültigen Nummern wird die Matrikelnummer auf den Wert 0 gesetzt.

Gibt die Matrikelnummer des Studenten als int-Wert zurück.

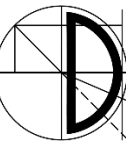




## Beispiel

Diesen Prozess bezeichnen wir üblicherweise als **information / data hiding**, das heißt als „Verstecken“ der Daten vor dem Benutzer

| Student                              |
|--------------------------------------|
| name:String<br>- nummer:integer      |
| setzeNummer(int)<br>liesNummer():int |



## Beispiel

Diesen Prozess bezeichnen wir üblicherweise als **information / data hiding**, das heißt als „Verstecken“ der Daten vor dem Benutzer

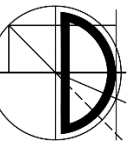
Das Binden von Daten und Methoden an eine spezielle Klasse erhebt die Objekte somit über ein reines Konstrukt zum Speichern von Daten hinaus.

Der Programmierer ist schon beim Entwurf seiner „Datenspeicher“ in der Lage, das spezifische Verhalten seiner Objekte (etwa auf Datenfehler) zu modellieren.

| Student                              |
|--------------------------------------|
| name:String<br>- nummer:integer      |
| setzeNummer(int)<br>liesNummer():int |



# One last thing...



Schon gewusst, dass einer der teuersten Software Bugs von einem Studenten verursacht wurde?

Der Student Robert Morris programmierte 1988 den ersten Computervorm. Aufgrund eines ungewollten Fehlers verbreitete der Wurm sich mit einer Wahrscheinlichkeit von 14/15 statt 1/15.

Zwischenzeitlich verursachte der Wurm etwa 10% des (noch jungen) Internetverkehrs und Schäden in Höhe von etwa 10 Mio. \$. Robert Morris wurde zu 3 Jahren Bewährung, sozialer Arbeit, etwa 10.000\$ Geldstrafe und zur Übernahme von Gerichtskosten in Höhe von etwa 150.000\$ verurteilt. Inzwischen ist er allerdings Informatik-Professor am MIT.



*"Error, no keyboard - press F1 to continue"*  
*-- early PC error message (BIOS)*

The image features a background of a classical building facade with columns and arches, likely a part of the Julius-Maximilians-Universität Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'J' and 'M' forming a square. The text 'Julius-Maximilians-' is in a smaller font above 'UNIVERSITÄT WÜRZBURG' which is in a large, bold, white sans-serif font.

Julius-Maximilians-  
**UNIVERSITÄT  
WÜRZBURG**

**Fragen?**

Vielen Dank für Ihre Aufmerksamkeit